Uttara University

Excellence in Higher Education and Research

Design and Implementation of a Smart Courier and Parcel Management System

Submitted By

SHARMIN KHATUN (ID: 2223081010)

TASNIM JAHAN MIM (ID: 2221081138)

Supervisor

DR. MD. OBAIDUR RAHAMAN

Professor, Department of CSE, Uttara University

In Fulfillment of the Requirements for the Degree of Bachelor of Science in Computer Science and Engineering
Uttara University, Dhaka, Bangladesh

27 October 2026

Design and Implementation of a Smart Courier and Parcel Management System

Submitted By

SHARMIN KHATUN
Student ID: 2223081010
E-mail: 2223081010@uttarauniversity.edu.bd

TASNIM JAHAN MIM
Student ID: 2221081138
E-mail: 2221081138@uttarauniversity.edu.bd

Supervisor

DR. MD. OBAIDUR RAHAMAN
Professor, Department of CSE, Uttara University

Department of Computer Science and Engineering
Uttara University
Dhaka, Bangladesh

27 October 2026

Abstract

Courier operations that coordinate parcels manually or with disconnected tools give customers little visibility, leave responsibility for each parcel unclear and keep no reliable record of status changes. This thesis designs, implements and evaluates the Smart Courier and Parcel Management System (SCPMS), a web application that gives one courier operator a shared, role-controlled record of every parcel and pushes changes to authorised users in real time.

SCPMS is built on MongoDB, Express, React and Node.js, with Socket.IO messaging and OpenStreetMap-based maps. Customers register with e-mail verification, book parcels and follow them on a map. Delivery agents confirm pickups by scanning the customer's QR code, advance parcels through a server-enforced six-state lifecycle and share their location. Administrators assign agents, manage accounts and export CSV and PDF reports, and anyone with a tracking code sees a reduced public view. Security measures include short-lived access tokens with rotating, reuse-detecting refresh cookies, role and ownership checks, account lockout, rate limiting and NoSQL-injection filtering.

A new black-box harness ran 66 API and Socket.IO test cases against the unmodified backend and an in-memory database; all 66 passed. Production dependencies have no known vulnerabilities. The evaluation also records fourteen findings, including 98 frontend lint errors, a permissive forward-skip rule in the lifecycle and a QR step that identifies parcels without proving handover. No usability study, load test or field trial was performed. The work shows that a secure, real-time courier system can be built with open-source technologies; usability and operational benefit remain to be established.

Keywords: courier management system; parcel tracking; real-time web applications; Socket.IO; role-based access control; MERN stack; web application security; software testing

27 October 2026
Department of Computer Science and Engineering
Uttara University

We officially attest that **SHARMIN KHATUN** (ID: 2223081010) and **TASNIM JAHAN MIM** (ID: 2221081138), students of the Department of Computer Science and Engineering (CSE) at Uttara University, have completed their thesis entitled "**Design and Implementation of a Smart Courier and Parcel Management System**" as a partial requirement for the Bachelor of Science degree in Computer Science and Engineering at Uttara University.

_____ Supervisor

Name: Dr. Md. Obaidur Rahaman
Designation: Professor
Department of Computer Science and Engineering
Uttara University

_____ Coordinator

Name: Md. Torikur Rahman
Designation: Assistant Professor
Department of Computer Science and Engineering
Uttara University

_____ Examiner

Name: .....
Designation: .....
Department of Computer Science and Engineering
Uttara University

_____ Head of the Department

Name: Dr. S M Nazmus Sadat
Designation: Assistant Professor
Department of Computer Science and Engineering
Uttara University

27 October 2026
Department of Computer Science and Engineering
Uttara University

We, the undersigned, declare that the project entitled "**Design and Implementation of a Smart Courier and Parcel Management System**" is the original work of **SHARMIN KHATUN** and **TASNIM JAHAN MIM**. The contributions of others, including software libraries, published literature and technical documentation, have been acknowledged through citations and references. This project was carried out under the supervision of **Dr. Md. Obaidur Rahaman** in the Department of Computer Science and Engineering at Uttara University, and it has not been submitted, in whole or in part, for any other degree.

Name: Sharmin Khatun
Student ID: 2223081010
Department of CSE, Uttara University

Name: Tasnim Jahan Mim
Student ID: 2221081138
Department of CSE, Uttara University

UTTARA UNIVERSITY

DECLARATION OF COPYRIGHT AND AFFIRMATION OF FAIR USE OF UNPUBLISHED RESEARCH

DESIGN AND IMPLEMENTATION OF A SMART COURIER AND PARCEL MANAGEMENT SYSTEM

We declare that Uttara University (UU) and the students jointly own the copyright to this dissertation.

© 2026 *Sharmin Khatun, Tasnim Jahan Mim* and Uttara University (UU). All rights reserved.

Without the previous written consent of the copyright holder, no portion of this unpublished study may be duplicated, saved in a retrieval system, or transmitted in any form or by any means, whether electronic, mechanical, photocopying, recording, or otherwise.

1. Others may use any content from this unpublished study in their writing as long as they give proper credit.
2. Print or electronic copies may be made and transmitted by UU or its library for academic and institutional purposes.
3. If other universities and research libraries request copies of this unpublished study, the UU library may create them, store them in a retrieval system, and provide them.

By signing this form, we attest that we have read and understood the UU Intellectual Property Right and Commercialization policy.

Affirmed by Tasnim Jahan Mim

_____ _____

Signature (on behalf of the team) Date

Dedication

To every delivery agent who carries a parcel the last mile,
and to every customer who waits at the door for it.

May each parcel arrive safely and on time,
with every step of its journey clear to the people it belongs to.

Acknowledgment

First and foremost, we are grateful to our parents and families, whose patience, encouragement and belief in us carried us through this work.

We express our sincere gratitude to our supervisor, Dr. Md. Obaidur Rahaman, for guidance, support and constructive feedback at every stage of this project. We also thank the members of the thesis committee and the faculty of the Department of Computer Science and Engineering, Uttara University, for their teaching and for their comments on our work. We thank our friends and classmates for their encouragement.

Finally, we acknowledge the maintainers of the open-source software on which the system depends, including Node.js, Express, MongoDB and Mongoose, React, Vite, Socket.IO, Leaflet, OpenStreetMap and the Open Source Routing Machine.

Table of Contents

Title Page . . . . . **ii**

Abstract . . . . . **iii**

Clearance Page . . . . . **iv**

Submission Page . . . . . **v**

Copyright . . . . . **vi**

Dedication . . . . . **vii**

Acknowledgment . . . . . **viii**

Table of Contents . . . . . **ix**

List of Tables . . . . . **xii**

List of Figures . . . . . **xiii**

List of Abbreviations . . . . . **xiv**

CHAPTER 1: Introduction **1**
1.1 Overview 1
1.2 Problem Statement 1
1.3 Motivation and Rationale 2
1.4 Aim and Objectives 2
1.5 Engineering Questions 3
1.6 Scope and Boundaries 3
1.7 Intended Users and Stakeholders 4
1.8 Significance and Contribution 4
1.9 Methodological Overview 5
1.10 Report Organization 5

CHAPTER 2: Literature Review **6**
2.1 Overview 6
2.2 Last-Mile Delivery and Courier Operations 6
2.3 Visibility, Tracking and Tracing 7
2.4 Existing Courier and Parcel Management Systems 7

2.5 Real-Time Web Communication . . . 8
2.6 Access Control and Web Application Security . . . 8
2.7 Maps, Geolocation and Machine-Readable Codes . . . 9
2.8 Comparison and Derived Requirements . . . 9
2.9 Summary . . . 10

CHAPTER 3: Methodology . . . 11
3.1 Overview . . . 11
3.2 Development Methodology . . . 11
3.3 Stakeholder and User Analysis . . . 12
3.4 Functional Requirements . . . 12
3.5 Non-Functional Requirements . . . 14
3.6 Use Cases . . . 14
3.7 System Architecture . . . 14
3.8 Database Design . . . 16
3.9 Parcel Lifecycle Model . . . 19
3.10 Interaction Design . . . 19
3.11 API and Real-Time Design . . . 21
3.12 Security and Access-Control Design . . . 23
3.13 Technology Selection and Justification . . . 24
3.14 Design Constraints and Assumptions . . . 24
3.15 Development Environment and Tools . . . 25
3.16 Project Structure . . . 25
3.17 Backend Bootstrap and Request Pipeline . . . 25
3.18 Authentication and Session Management . . . 26
3.19 Authorisation . . . 28
3.20 Parcel Booking and Lifecycle . . . 29
3.21 Real-Time Layer . . . 31
3.22 Location, Maps and Estimated Arrival . . . 31
3.23 QR-Assisted Pickup . . . 33
3.24 Notifications . . . 35
3.25 Analytics, Export and User Management . . . 35
3.26 Frontend Implementation and Internationalisation . . . 35
3.27 Validation, Error Handling and Hardening . . . 37
3.28 Deployment and Execution . . . 38
3.29 Implementation Challenges . . . 38
3.30 Summary . . . 39

CHAPTER 4: Analysis, Results and Discussion . . . 40
4.1 Overview . . . 40
4.2 Evaluation Objectives . . . 40
4.3 Test Environment . . . 40
4.4 Testing Strategy . . . 41
4.5 Static Analysis and Build Results . . . 42
4.6 Functional and Integration Test Results . . . 43
4.7 Security and Access-Control Test Results . . . 44
4.8 Real-Time Behaviour . . . 44

4.9 Performance Observations . . . 45
4.10 Visual Verification of the Running System . . . 46
4.11 Defects and Findings . . . 46
4.12 Requirements Traceability . . . 47
4.13 Discussion . . . 49
4.14 Summary . . . 49

CHAPTER 5: Limitations . . . 51
5.1 Overview . . . 51
5.2 Limitations of the System and the Evaluation . . . 51
5.3 Evaluations Not Performed . . . 51
5.4 Summary . . . 52

CHAPTER 6: Conclusion . . . 53
6.1 Overview . . . 53
6.2 Summary of the Project . . . 53
6.3 Achievement of Objectives . . . 53
6.4 Answers to the Engineering Questions . . . 53
6.5 Key Technical Contributions . . . 54
6.6 Lessons Learned . . . 55
6.7 Recommendations for Future Work . . . 55
6.8 Final Conclusion . . . 56

References . . . 57

List of Tables

1.1 Objectives, implementing components and evaluation evidence 3

2.1 Qualitative comparison of solution categories against the needs of a small courier operator 10

3.1 User roles, goals and granted capabilities 12
3.2 Functional requirements 13
3.3 Non-functional requirements and verification methods 16
3.4 Descriptions of the principal use cases 17
3.5 REST API groups 21
3.6 Socket.IO events 22
3.7 Threats, controls and OWASP Top 10 (2021) categories 23
3.8 Technology choices and justification 24
3.9 Development and evaluation environment 25
3.10 Organisation of the repository 26
3.11 Order of checks applied to every requested status change 30

4.1 Environment of the executed evaluation 41
4.2 ESLint findings for the frontend 42
4.3 Summary of executed black-box tests by area 43
4.4 Locally measured latencies (sequential requests, loop-back interface, in-memory database) 45
4.5 Defects and findings 46
4.6 Requirements traceability matrix 48

6.1 Assessment of objectives against evidence 54

List of Figures

3.1 Use-case diagram of SCPMS 15
3.2 System context diagram 15
3.3 Layered architecture of SCPMS 17
3.4 Entity–relationship diagram of the three collections 18
3.5 Parcel lifecycle state machine as enforced by the backend 19
3.6 Sequence diagram: booking, assignment and QR-assisted pickup 20
3.7 Sequence diagram: live agent location 21
3.8 Scoping of real-time events through Socket.IO rooms 22
3.9 Customer registration page (running system) 27
3.10 Sequence diagram: sign-in, silent refresh with rotation, and reuse detection 28
3.11 Customer dashboard with booking form and parcel list (running system, demonstration data) 29
3.12 Agent parcel page with lifecycle progress, status update and location controls (running system) 30
3.13 Administrator agent-tracking page receiving a live position from a connected agent (running system; position simulated) 32
3.14 Public tracking by code with OSM map and OSRM route; no customer data is shown (running system) 33
3.15 Customer parcel page with agent card, distance and heuristic ETA (running system). The COD amount is shown with a "$" sign, a currency-symbol defect recorded in Chapter 4 34
3.16 QR-assisted pickup (running system). The confirmation page is normally reached by scanning; for the screenshot it was opened directly by URL . . 34
3.17 Administrator dashboard: indicators, user management, bookings with assignment and CSV/PDF export (running system, demonstration data) . . 36
3.18 Landing page with public tracking entry in both interface languages (running system) 36
3.19 Delivery-agent dashboard (running system, demonstration data) 37
3.20 Deployment view as configured in the repository 38

List of Abbreviations

API	Application Programming Interface
BN	Bangla (Bengali) language
COD	Cash on Delivery
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
CSV	Comma-Separated Values
ER	Entity–Relationship
ETA	Estimated Time of Arrival
FR	Functional Requirement
HTTP	Hypertext Transfer Protocol
IDOR	Insecure Direct Object Reference
JSON	JavaScript Object Notation
JWT	JSON Web Token
MERN	MongoDB, Express, React, Node.js
NFR	Non-Functional Requirement
ODM	Object Document Mapper
OSM	OpenStreetMap
OSRM	Open Source Routing Machine
OTP	One-Time Passcode
OWASP	Open Worldwide Application Security Project
PDF	Portable Document Format
QR	Quick Response (code)
RBAC	Role-Based Access Control
REST	Representational State Transfer
SCPMS	Smart Courier and Parcel Management System
SMTP	Simple Mail Transfer Protocol
SPA	Single-Page Application
TTL	Time to Live
UI	User Interface
UML	Unified Modeling Language

Chapter 1

Introduction

1.1 Overview

Parcel delivery has become a routine part of commerce and daily life. The growth of e-commerce has greatly increased the number of direct-to-consumer deliveries and, with them, the challenges of the "last mile": the final leg in which a parcel is collected from a sender and handed to a recipient [1, 2]. Reviews of last-mile research describe this leg as one of the least efficient and most costly stages of the delivery chain, and identify information sharing, routing and coordination between actors as recurring problems [2, 3, 4]. In cities the difficulty is compounded by congestion and by customers' desire for fast delivery, sometimes on the same day [1].

For a courier operator the practical consequence is a coordination problem. Customers want to know where their parcel is; dispatchers need to know which agent holds which parcel; and delivery agents need an unambiguous list of assignments and a simple way to record progress. When this information is spread across telephone calls, paper records and informal messaging, every party works with an incomplete and delayed picture. The literature on supply-chain visibility makes the same point at a larger scale: the ability to *track* a product forward and *trace* its history backward depends on capturing status events in a shared, structured record [5].

This thesis presents the design, implementation and evaluation of the **Smart Courier and Parcel Management System (SCPMS)**, a web application that gives a single courier operator one shared, role-controlled record of every parcel and propagates changes to that record to the interested parties in real time. The system is built on the MERN stack (MongoDB, Express, React and Node.js) with Socket.IO for real-time messaging and OpenStreetMap-based maps. It supports three authenticated roles, namely customer, delivery agent and administrator, together with an unauthenticated public tracking page.

1.2 Problem Statement

The project addresses the following problem, as identified in the project proposal and progress report and confirmed against the scope of the implemented system:

> *Small and medium courier operations that coordinate parcels manually, or with disconnected tools, cannot give customers timely visibility of their parcels, cannot reliably control which agent is responsible for which parcel, and lack a consolidated operational view for administrators. Status changes are communicated late or inconsistently, and there is no trustworthy record*

of who changed a parcel's status and when.

Four specific deficiencies follow from this statement:

1. **Visibility:** customers cannot see the current status or approximate position of a parcel without contacting the operator.
2. **Coordination and responsibility:** assignment of parcels to agents, and the rules for who may change a parcel's state, are informal.
3. **Accountability:** the history of status changes is not recorded with the identity of the actor.
4. **Operational insight:** administrators lack basic indicators (bookings, failed deliveries, cash-on-delivery totals) and exportable records.

Because the system handles personal data (names, addresses, telephone numbers) and controls operational state, any solution must also be **secure**: users must only see and change what their role permits, and public tracking must not expose personal data.

1.3 Motivation and Rationale

Commercial carriers operate their own tracking platforms [6, 7], and several published tracking systems are built around dedicated tracking hardware installed in each vehicle [8, 9]. A small operator, however, needs neither proprietary infrastructure nor dedicated hardware: delivery agents already carry smartphones with a browser, a camera and a location sensor. Web standards such as the Geolocation API [10] and the WebSocket protocol [11], together with open map data [12] and open routing engines [13], make it possible to build live tracking and camera-based identification with commodity devices and open-source software.

The project was therefore motivated by a practical question: whether a small team can build, with widely used open-source web technologies, a courier system that combines role-based coordination, real-time visibility and secure public tracking in one coherent and verifiable application. The local context of the authors (Dhaka, Bangladesh) shaped several requirements, including Bangladeshi mobile-number validation, cash-on-delivery (COD) support and a Bangla user interface.

1.4 Aim and Objectives

Aim. To design, implement and evaluate a secure, role-based web system that manages the parcel lifecycle of a courier operator and provides real-time parcel and agent visibility to authorised users.

Specific objectives. Table 1.1 lists the objectives and links each one to the implementation component that realises it and the evidence used to evaluate it.

Table 1.1: Objectives, implementing components and evaluation evidence

ID	Objective	Implementing component	Evaluation evidence
O1	Provide secure registration, authentication and session management for three roles	Auth controller, session service, JWT utilities (Section 3.18)	Tests T05–T22, T61–T62
O2	Enforce role-based and ownership-based access control on every operation	`requireRoles`, ownership checks, route guards (Section 3.19)	Tests T23–T25, T30–T32, T39–T41, T50, T58
O3	Manage the parcel lifecycle from booking to delivery with server-enforced transitions and an audit trail	Parcel model and controller (Section 3.20)	Tests T26–T29, T42–T56
O4	Propagate status and location changes in real time to authorised watchers only, and offer public tracking by code	Socket.IO rooms, public tracking endpoint (Section 3.21)	Tests T34–T35, T43, T49, T51
O5	Support QR-assisted parcel identification at pickup and map-based route display	QR pages, Leaflet/OSRM maps (Sections 3.22–3.23)	Screenshots; tests T44; manual inspection
O6	Give administrators operational indicators, user management and CSV/PDF export	Admin panel, analytics controller (Section 3.25)	Tests T24, T57–T62

1.5 Engineering Questions

The project is an applied engineering project rather than an experimental study. Three engineering questions frame the evaluation:

EQ1 How can a web application give customers, delivery agents and administrators consistent, role-appropriate visibility of a shared parcel lifecycle?

EQ2 How can live status and location changes be propagated in real time without disclosing information to unauthorised parties, including anonymous public viewers?

EQ3 To what extent does the implemented system satisfy its functional and security requirements, as measured by executable tests against the unmodified code?

EQ1 and EQ2 are answered through the design and implementation (Chapter 3). EQ3 is answered by the test evidence in Chapter 4. Chapter 6 returns to all three.

1.6 Scope and Boundaries

In scope. Customer self-registration with e-mail verification; administrator-managed agent accounts; parcel booking; agent assignment; a six-state lifecycle (Pending, Assigned, Picked Up, In Transit, Delivered, Failed); real-time status and location updates;

public tracking by code; QR-assisted pickup; e-mail notifications; basic operational metrics; CSV and PDF export; an English and Bangla interface; and deployment configuration for Render and Vercel/Netlify.

Out of scope. Multiple branches or hubs; warehouse management; route optimisation; pricing and payment processing (COD amounts are recorded, not collected electronically); SMS notifications; native mobile applications; delay prediction; and integration with external carriers. Some of these were mentioned in early project documents but were not implemented and are not claimed in this report.

Evaluation boundary. Evaluation was carried out on a local machine against an in-memory database. No usability study, field trial with real agents or load test against a live deployment was performed; these are identified as future work.

1.7 Intended Users and Stakeholders

- **Customers** (senders) who book parcels, follow them and receive notifications.
- **Delivery agents** who pick up and deliver parcels and share their location while working.
- **Administrators** (dispatchers or managers of the courier operator) who assign agents, manage accounts and monitor operations.
- **Recipients and other public viewers** who hold a tracking code and want to see a parcel's status without an account.
- **The operator** as an organisation, which needs reliable records and protection of customer data.

1.8 Significance and Contribution

The contribution of this work is the *design, integration and evidence-based evaluation* of a complete courier management system. It does not propose a new algorithm. Specifically, the thesis contributes:

1. a working, documented system that integrates role-based coordination, a server-enforced parcel lifecycle with an audit trail, and real-time tracking for one courier operator;
2. a real-time design in which every Socket.IO event is delivered through scoped rooms, so that anonymous viewers receive updates only for a parcel whose tracking code they have proven;
3. a session design that combines short-lived access tokens with rotating, reuse-detecting refresh cookies, applied and tested in an undergraduate-scale system;
4. a reproducible evaluation package: 66 executable black-box tests, static-analysis results and screenshots of the running system, together with an honest account of defects and limitations.

1.9 Methodological Overview

The project followed an iterative and incremental development process [14, 15]: requirements were derived from the courier workflow and the proposal, the core modules (authentication, lifecycle and tracking) were built first, and notifications, analytics, the bilingual interface and security hardening were added in later increments. The evaluation combines static analysis, a production build, a dependency audit, black-box API and Socket.IO tests against the unmodified backend, and local latency sampling. Chapter 3 describes the method in detail.

1.10 Report Organization

Chapter 2 reviews literature on last-mile logistics, tracking and visibility, comparable courier systems, real-time web communication, access control and web security. Chapter 3 describes the methodology: the requirements, models and design, and their implementation, with screenshots of the running system. Chapter 4 analyses and discusses the results of the testing and evaluation, including the requirements traceability. Chapter 5 states the limitations of the system and of its evaluation. Chapter 6 summarises the work, answers the engineering questions and outlines future work.

Chapter 2

Literature Review

2.1 Overview

This chapter reviews the literature and technical sources that inform the design of SCPMS. It begins with the operational context of last-mile courier logistics (Section 2.2) and the concept of visibility through tracking and tracing (Section 2.3). It then examines comparable courier management systems (Section 2.4) and the technical foundations of the solution: real-time web communication (Section 2.5), access control and web security (Section 2.6), and maps, geolocation and machine-readable codes (Section 2.7). Section 2.8 compares the approaches and derives the requirements carried into Chapter 3.

Peer-reviewed literature is used to establish context and design principles. Official standards and documentation are used only to support specific technical facts about the technologies used, and the two kinds of source are kept distinct.

2.2 Last-Mile Delivery and Courier Operations

Boysen, Fedtke and Schwerdfeger survey last-mile delivery concepts from an operational-research perspective, ranging from conventional attended home delivery to parcel lockers, crowd-sourced delivery, drones and autonomous robots [2]. They show that every concept raises its own planning and control problems, such as assignment, routing and scheduling, and that all of them rely on accurate information about orders, vehicles and customers. Ranieri et al. review last-mile innovations aimed at reducing external costs and group them into innovative vehicles, proximity stations, collaborative urban logistics, transport-management and routing optimisation, and public-policy measures. They also note that the last mile is the least efficient stage of the supply chain and can account for up to 28% of total delivery cost [3]. Olsson, Hellström and Pålsson propose a framework that organises last-mile research, highlighting the growing attention given to technology and information-sharing as enablers [4]. Savelsbergh and Van Woensel discuss city logistics. They identify the growth of e-commerce, which has sharply increased direct-to-consumer deliveries, and the desire for speed, such as same-day delivery, as drivers of change in urban delivery, and note that information systems shared between the parties can improve its reliability and visibility [1].

Two observations from this literature shape the present project. First, the last mile is information-intensive: dispatchers must know which parcels are pending and which agent is responsible for each. Second, the operational-research survey reviewed here treats the last mile mainly as a set of planning and optimisation problems (assignment, routing, scheduling) [2]; the *information system* that captures and distributes operational state lies

outside its scope. For a small operator without an existing platform, that information system is the first need. SCPMS addresses this layer and leaves optimisation as future work.

2.3 Visibility, Tracking and Tracing

Musa, Gunasekaran and Yusuf define product visibility as the ability to know the identity, location and status of a product as it moves through the supply chain. They distinguish *tracking* (following an item forward towards the customer) from *tracing* (reconstructing its history) [5]. Their review, based on academic and trade literature and an industry survey, finds that visibility systems depend on capturing identification and status events in a shared record, and that the benefits include better customer service and accountability. Both functions appear in SCPMS: the current status and location of a parcel support tracking, and the recorded status history (status, time, actor, role) supports tracing.

Delivery-confirmation research shows a related concern. Wibbeling and Schneiders describe an electronic proof-of-delivery project in which the handover at the point of delivery is captured electronically rather than on paper [16]. The underlying idea is that a handover should be recorded at the moment and place it happens, by the party performing it. SCPMS applies a lightweight form of this idea at *pickup*: the agent scans a QR code shown by the customer, and the server accepts the resulting status change only from the agent assigned to that parcel. The QR code is an identification aid. It does not provide cryptographic proof of handover, a limitation discussed in Chapter 5.

2.4 Existing Courier and Parcel Management Systems

Existing solutions fall into three broad categories.

Proprietary carrier platforms. Large carriers and regional courier companies operate their own booking and tracking platforms, for example DHL's shipment-tracking service [6] and, in Bangladesh, Pathao Courier's merchant booking with tracking by consignment number [7]. Their internal architectures, data models and access-control rules are not published, so they cannot be examined or reproduced academically, and they are not available to a small operator as a self-hosted system. This report therefore does not make comparative claims about the behaviour or performance of any named commercial platform.

Optimisation-oriented research systems. Much of the academic literature reviewed above concentrates on planning models and algorithms [2, 1]. These works assume that order and position data are available and focus on how to use them, not on how a small organisation should capture and share them securely.

Web-based courier management applications. A further category consists of web applications that implement booking, status updates and tracking for a single organisation, often as student or small-team projects. Parcel Pulse is a recent example: a three-tier application (HTML/CSS/JavaScript front end, PHP and MySQL) that separates customers, staff and administrators and supports booking and shipment tracking [17]. Its published abstract reports no measured evaluation, and it does not state how status changes reach

users or how sessions, ownership checks and public tracking pages are protected. Because such security and evaluation details are not available for the systems reviewed here, this thesis documents its own test evidence in detail.

2.5 Real-Time Web Communication

The architectural style of most web APIs is Representational State Transfer (REST), in which clients manipulate resources through a uniform interface of stateless requests [18]. REST suits operations such as booking or assigning a parcel. On its own, however, it gives the client no means of learning about changes it did not initiate: a client must poll repeatedly, which wastes bandwidth and delays updates by up to one polling interval.

The WebSocket protocol, standardised in RFC 6455, establishes a persistent, full-duplex channel over a single TCP connection after an HTTP upgrade handshake [11]. Pimentel and Nickerson studied WebSocket for communicating and displaying real-time sensor data in the browser and found it well suited to pushing frequent small updates to many clients, compared with HTTP polling approaches [19]. Node.js complements this model: its single-threaded, event-driven, non-blocking I/O handles many concurrent, mostly idle connections efficiently [20]. Socket.IO builds on WebSocket with automatic reconnection and *rooms*, named groups of connections to which the server can address an event [21]. Rooms are central to SCPMS: they allow an event to be delivered only to the parties entitled to it (Section 3.11).

2.6 Access Control and Web Application Security

Role-based access control. Sandhu et al. formalised role-based access control (RBAC), in which permissions are attached to roles and users acquire permissions through role membership [22]. RBAC simplifies administration because policies are expressed per role rather than per user. Pure RBAC is not sufficient for a courier system, however: a customer may read *their own* parcels but not those of other customers, and an agent may update only parcels *assigned to them*. Role checks must therefore be combined with ownership checks on each object.

Common vulnerabilities. The OWASP Top 10 (2021) ranks *Broken Access Control* as the most serious web-application risk, followed by cryptographic failures and injection, and lists identification and authentication failures among the top ten [23]. Insecure direct object references, where a user retrieves another user's record by changing an identifier, are a typical example of broken access control. For document databases, Ron, Shulman-Peleg and Puzanov show that NoSQL databases are also vulnerable to injection. A JSON request body containing query operators (for example `{"$ne": null}`) can change the meaning of a query if user input is passed to the database unchecked [24].

Authentication and sessions. JSON Web Tokens (JWT, RFC 7519) are a compact, signed format for transferring claims between parties [25]. They let a stateless API verify a user's identity and role on each request, but they cannot be withdrawn before they expire unless the server keeps additional state. Long-lived tokens stored in JavaScript-accessible storage can be stolen through cross-site scripting. Cookies marked `HttpOnly` cannot be read by scripts [26], but cookie-authenticated endpoints then need protection against

cross-site request forgery (CSRF) [27]. The IETF's current best practice for OAuth 2.0 requires that refresh tokens issued to public clients be either sender-constrained or *rotated*, with reuse of an old refresh token treated as a sign of theft [28]. SCPMS is not an OAuth system, but its session design adopts the same rotation-with-reuse-detection principle (Section 3.18).

Passwords. Passwords should be stored with a deliberately slow, salted hash function. Bcrypt, introduced by Provos and Mazières, has an adjustable cost factor so that hashing can be made more expensive as hardware improves [29]. NIST SP 800-63B requires memorised secrets of at least eight characters and rate limiting of failed authentication attempts. It recommends against imposing composition rules and recommends screening new passwords against lists of compromised values [30]. Section 3.12 compares the SCPMS password policy with these recommendations.

2.7 Maps, Geolocation and Machine-Readable Codes

OpenStreetMap (OSM) is a collaboratively edited, openly licensed map of the world [12]. Its data can be used without licence fees, which makes it attractive for small operators. Leaflet is a lightweight JavaScript library for interactive maps that can display OSM tiles [31]. The Open Source Routing Machine (OSRM) computes shortest road routes on OSM data quickly enough for interactive use [13]. Converting free-text addresses to coordinates (geocoding) can be done with OSM-based services. The public Nominatim service, however, has a strict usage policy, including a maximum of one request per second and a requirement to identify the application [32]. Public instances of such services are therefore unsuitable for high-volume production use without caching or a self-hosted instance.

In the browser, the W3C Geolocation API provides the device's position, with user permission, either once or as a continuous watch [10]. QR codes (ISO/IEC 18004) encode short strings in a two-dimensional symbol that phone cameras can read reliably [33]. In browsers without a native barcode detector, the open-source ZXing library can decode them from a video stream [34]. Together these allow an ordinary phone browser to act as a location reporter and a scanner, with no dedicated tracking hardware.

2.8 Comparison and Derived Requirements

Table 2.1 compares the three categories of existing solution described in Section 2.4 with the needs of a small courier operator identified in Chapter 1. The comparison is qualitative and based on the cited sources. It is not a measured benchmark.

The literature and the comparison lead to the following high-level requirements, which Chapter 3 refines into functional and non-functional requirements:

R1. One structured, shared record of each parcel, including a status history with actor and time [5].

R2. Role-based access combined with object ownership checks, addressing broken access control [22, 23].

R3. Push-based propagation of changes to authorised clients only [11, 19].

Table 2.1: Qualitative comparison of solution categories against the needs of a small courier operator

Need	Proprietary carrier platforms	Optimisation research	Published web CMS example [17]
Self-hostable by a small operator	No	Not applicable (models)	Yes
Shared parcel record with roles	Yes (internal, unpublished)	Assumed	Yes (three roles)
Server-enforced lifecycle and audit trail	Unknown (not published)	Not addressed	Not described
Push-based real-time updates	Unknown	Not addressed	Not described
Privacy-preserving public tracking	Unknown	Not addressed	Not described
Documented security and test evidence	Not available	Not applicable	Not reported

R4. Sessions that resist token theft and can be revoked [26, 28].

R5. Input handling that prevents NoSQL injection [24].

R6. Map display and location capture with open data and standard browser APIs, within the usage limits of public services [12, 10, 32].

R7. Camera-based identification of parcels at handover, with commodity phones [33, 16].

2.9 Summary

The reviewed literature shows that last-mile delivery is costly and information-intensive, and that visibility depends on capturing status events in a shared record. The operational-research literature concentrates on planning and optimisation, while the information system that a small operator needs first lies outside its scope. The published web-based courier application reviewed here reports neither its security design nor a measured evaluation. The technical sources describe mature, openly available building blocks: WebSocket and rooms for push updates, RBAC with ownership checks, rotating refresh tokens, slow password hashing, OSM-based maps and QR codes. The next chapter turns these into requirements and a design for SCPMS.

Chapter 3

Methodology

3.1 Overview

This chapter describes the method used to build SCPMS: how the system was analysed and designed, and how the design was implemented. It presents the development methodology (Section 3.2), the stakeholders and requirements (Sections 3.3–3.5), the use-case model (Section 3.6), and the system architecture, data model, lifecycle and interaction designs (Sections 3.7–3.10). It then covers the API and real-time design, the security design and the technology choices (Sections 3.11–3.13). It then describes the implementation: the development environment and project structure (Sections 3.15–3.16), the backend modules (Sections 3.17–3.25), the frontend (Section 3.26), cross-cutting validation and hardening (Section 3.27), deployment (Section 3.28) and the main challenges (Section 3.29).

All models in this chapter were derived from the implemented source code and checked against it. Where the design deviates from earlier project documents, the implemented design is described. File paths refer to the project repository. The behaviour of the code is described in prose, tables and diagrams; no source code is reproduced.

All screenshots in this chapter were captured from the *running application*, with the unmodified backend and the Vite development server, on a local machine. The database was a temporary in-memory MongoDB filled with demonstration data through the public API. The agent's position was a fixed, simulated browser location in Dhaka (23.8223 N, 90.3654 E). The capture script is included in the repository (`report/evidence/screenshots`). No screenshot is a mock-up.

3.2 Development Methodology

The project used an iterative and incremental process [14, 35]. Each increment produced working software that was exercised before the next was started, which suited a two-person team with a fixed defense date. The increments, reconstructed from the progress report (22 July 2026), the security review records and the code, were:

1. **Planning and analysis** (late April to May 2026): problem definition, literature review, and identification of roles and lifecycle states.
2. **Design** (May 2026): data model, interface wireframes for the three roles, and the choice of the MERN stack with Socket.IO.

3. **Core implementation** (May to July 2026): authentication, parcel booking, assignment, lifecycle, real-time tracking and QR-assisted pickup.
4. **Extension** (July to October 2026): e-mail notifications, analytics and export, the Bangla interface, the landing page and deployment configuration.
5. **Security hardening** (two rounds, July and October 2026): fixes for access-control, session and injection weaknesses found in code review (Section 3.12).
6. **Evaluation and documentation** (October 2026): the test harness, static analysis and this report.

The phases above are reconstructed from the dated project documents.

3.3 Stakeholder and User Analysis

Table 3.1 summarises the user roles, their goals and the access the system grants them.

Table 3.1: User roles, goals and granted capabilities

Role	Goals	Capabilities in SCPMS
Customer	Send parcels and know their status without phoning the operator	Self-register (e-mail verified); book parcels; list and view *own* parcels; show a parcel QR code; follow a parcel on a map; receive status e-mails
Delivery agent	Know assigned work; record progress quickly in the field	Account created by an administrator (invitation); list *assigned* parcels; scan the QR code at pickup; move parcels forward or mark them failed; share live location
Administrator	Coordinate agents; keep accurate records; monitor operations	List all users and parcels; add, disable and re-enable accounts; assign and reassign agents; correct non-final statuses; delete parcels; view metrics; export CSV and PDF; track agents on a map
Public viewer	Check a parcel's status without an account	View a reduced parcel record (no customer identity, contact details or notes) and live updates for a known tracking code

3.4 Functional Requirements

Table 3.2 lists the functional requirements. They were consolidated from the project proposal, the progress report and the implemented behaviour. Requirements that appeared in early documents but were not implemented (for example SMS notifications, revenue reports or a manager role) are deliberately excluded.

Table 3.2: Functional requirements

ID	Requirement	Actor(s)
FR-01	The system shall let a visitor register as a customer with name, e-mail, Bangladeshi mobile number and a password satisfying the password policy, after accepting the terms. Any role supplied by the client shall be ignored.	Customer
FR-02	The system shall authenticate users by e-mail and password, refuse sign-in to unverified or disabled accounts, and lock an account for 15 minutes after five consecutive failed attempts.	All
FR-03	The system shall maintain sessions with a short-lived access token and a rotating refresh token, and support sign-out.	All
FR-04	The system shall restrict every operation according to the user's role and, where relevant, ownership of or assignment to the parcel.	All
FR-05	Administrators shall create delivery-agent accounts, which are activated by an invitation link, and shall disable or re-enable any account other than their own.	Administrator
FR-06	Customers shall book a parcel by entering pickup and delivery addresses, size, type, payment type, COD amount (0–500,000) and optional notes. Invalid input shall be rejected.	Customer
FR-07	The system shall assign each parcel a unique, hard-to-guess tracking code.	System
FR-08	The parcel list shall show customers their own parcels, agents their assigned parcels and administrators all parcels (optionally filtered by agent).	All roles
FR-09	Anyone holding a tracking code shall be able to view the parcel's status, addresses, current location, agent name and status timeline, without personal data of the customer.	Public viewer
FR-10	Administrators shall assign an active, activated agent to a parcel and may reassign it while it is Pending or Assigned.	Administrator
FR-11	Parcel status shall follow the lifecycle rules of Figure 3.5: agents move assigned parcels forward or mark them Failed, administrators may correct non-final parcels, and final states are locked.	Agent, Administrator

ID	Requirement	Actor(s)
FR-12	Assigned agents shall share their location. The system shall store the parcel's current location and push location and status changes to authorised watchers in real time.	Agent, System
FR-13	The system shall record every status change with the new status, time, acting user and role.	System
FR-14	Administrators shall see the number of bookings today, the number of failed deliveries and the total COD collected from delivered parcels.	Administrator
FR-15	Administrators shall export the parcel list as CSV and PDF.	Administrator
FR-16	Administrators shall delete parcels.	Administrator
FR-17	Users shall reset a forgotten password through a single-use, time-limited e-mail link.	All
FR-18	Customers shall display a QR code for each parcel, and agents shall scan it (or enter it manually) to open the pickup confirmation for that parcel.	Customer, Agent
FR-19	The system shall e-mail the customer when a parcel's status changes.	System
FR-20	The interface shall be available in English and Bangla.	All
FR-21	Parcel pages shall show pickup and delivery points and a road route on a map, together with the agent's live position when available.	All

3.5 Non-Functional Requirements

The non-functional requirements (Table 3.3) follow the quality characteristics of ISO/IEC 25010 [36]. Each is stated with the way it was verified, in the spirit of ISO/IEC/IEEE 29148, which asks that requirements be verifiable [37].

3.6 Use Cases

Figure 3.1 shows the use-case model. Table 3.4 describes the four use cases that carry the main workflow.

3.7 System Architecture

Context. Figure 3.2 places SCPMS in its environment. Four kinds of user interact with it. It depends on MongoDB for persistence, an SMTP server for e-mail and OSM-based services for geocoding, map tiles and routing.

Layers. SCPMS is a client–server single-page application (Figure 3.3). The React client is delivered as static files and talks to the backend in two ways. Ordinary operations use

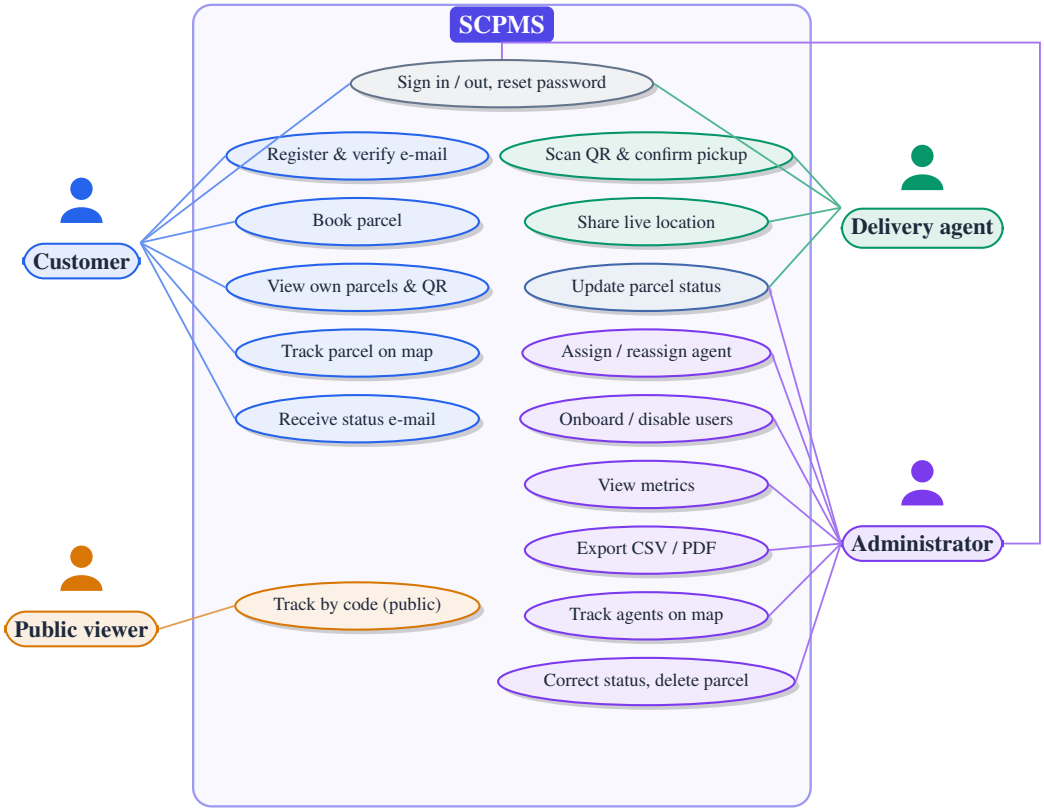


Figure 3.1: Use-case diagram of SCPMS

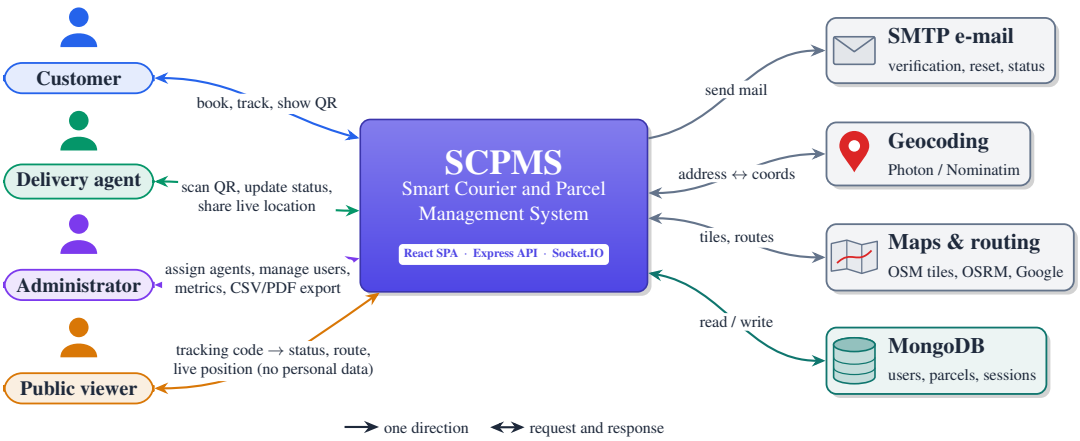


Figure 3.2: System context diagram

Table 3.3: Non-functional requirements and verification methods

Quality	Requirement	Verification
Security	Passwords hashed with bcrypt; credentials never returned; access tokens short-lived; refresh tokens stored only as hashes; brute-force and enumeration resisted; NoSQL operators stripped from input; security headers set	Tests T02, T11–T22, T62–T63, T65–T66; code review
Privacy	Public tracking and public sockets expose no customer identity, contact details, notes or other customers' data	Tests T34, T43, T51
Responsiveness	Status and location changes reach connected, authorised clients without page refresh	Tests T43, T49, T51; latency sampling (Section 4.9)
Robustness	Malformed JSON, invalid identifiers and unknown routes produce 4xx responses, not crashes or stack traces	Tests T03, T04, T33
Usability	Responsive layout; English and Bangla; manual fallback when the camera is unavailable	Screenshots; manual inspection (no user study performed)
Maintainability	Layered backend (routes, controllers, models, services, utilities); shared constants for enums	Code inspection; lint results
Deployability	Configuration through environment variables; production refuses to start without a strong JWT secret and an origin allow-list; health endpoint	Code inspection; test T01

REST over HTTPS on the same origin (`/api`), proxied to the backend so that the refresh cookie is treated as first-party. Real-time events use a Socket.IO connection opened directly to the backend. The backend is a single Node.js process with an Express application and a Socket.IO server sharing one HTTP server. Inside it, requests pass through a fixed middleware chain, then route modules that bind URLs to role guards, then controllers that validate input and apply business rules, and finally Mongoose models. Cross-cutting concerns live in services (sessions, socket handling, room-scoped emission) and utilities (JWT, random tokens, validation, mail).

Rationale. A layered, MVC-inspired backend keeps HTTP concerns, business rules and persistence separate, so that rules such as the lifecycle transition check exist in one place (the controller) and are applied to every client. Running REST and Socket.IO in one process lets controllers emit real-time events directly after a successful database write, without a message broker. The cost is that horizontal scaling would require a shared adapter for Socket.IO rooms, a limitation discussed in Chapter 5.

3.8 Database Design

SCPMS stores its data in three MongoDB collections defined with Mongoose schemas (Figure 3.4).

Table 3.4: Descriptions of the principal use cases

Use case	Description
UC1 Book parcel	*Actor:* customer (signed in, verified). *Flow:* enters addresses, size, type, payment and COD amount; system validates, generates a tracking code, stores the parcel as Pending and notifies administrators in real time. *Result:* parcel listed with its tracking code and QR code. *Alternative:* invalid input returns an error message and nothing is stored.
UC2 Assign agent	*Actor:* administrator. *Flow:* selects an active agent for a Pending or Assigned parcel; system stores the assignment, appends Assigned to the history and pushes the update to the customer, agent and watchers. *Alternative:* agent inactive or invitation pending, or parcel already picked up: request refused.
UC3 Confirm pickup	*Actor:* assigned agent. *Flow:* scans the customer's QR code (or pastes the link or id); confirmation page shows parcel details; agent confirms; system sets Picked Up if the caller is the assigned agent. *Alternative:* parcel not assigned to the caller: 403 and explanatory message.
UC4 Track parcel	*Actor:* customer or public viewer. *Flow:* opens the parcel page or enters a tracking code; system returns the parcel (reduced view for public) and subscribes the browser to the parcel's real-time room; map shows route and agent position as updates arrive. *Alternative:* unknown code: "not found".

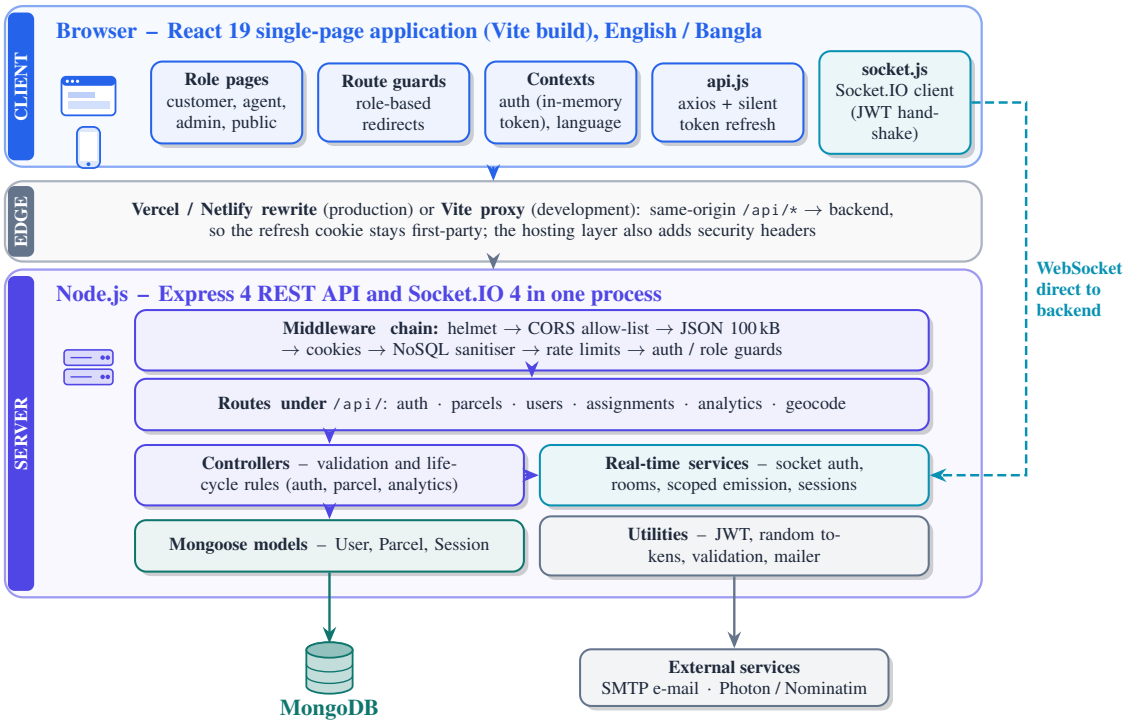


Figure 3.3: Layered architecture of SCPMS

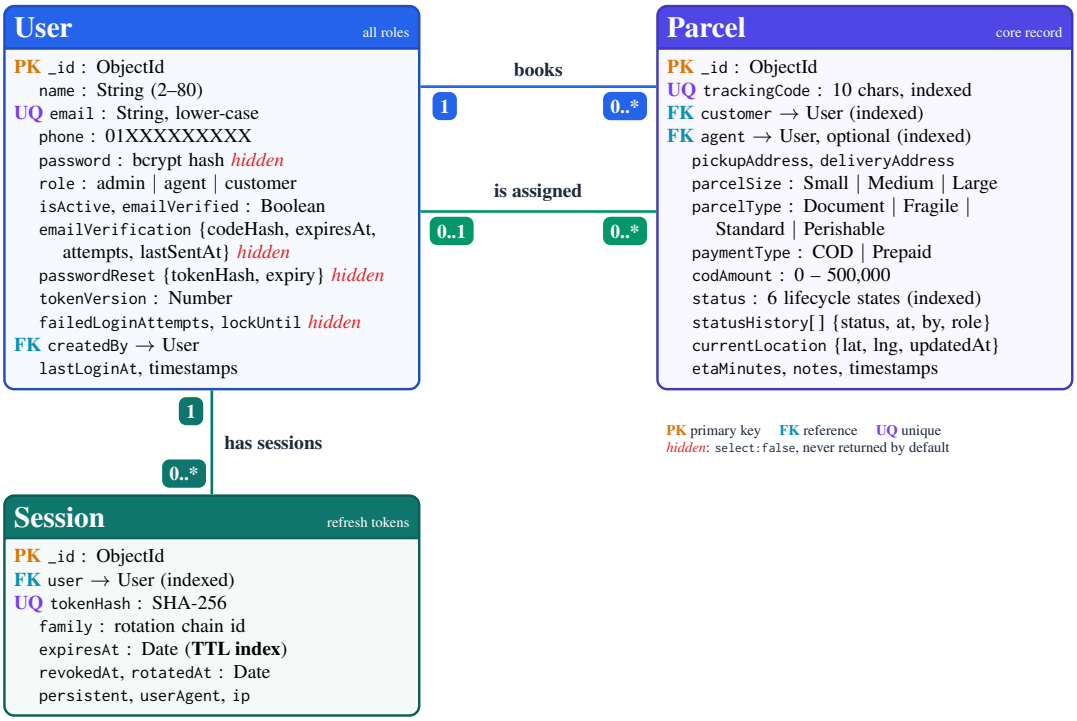


Figure 3.4: Entity–relationship diagram of the three collections (derived from `backend/src/models`)

- **User** holds identity and credentials for all roles. Secret fields (password hash, verification-code hash, reset-token hash, lockout counters) are marked `select: false`, so they are never loaded unless a query asks for them explicitly. The `tokenVersion` counter supports immediate revocation of access tokens. `createdBy` records which administrator created an agent.
- **Parcel** references its customer and (optionally) its agent. It embeds its `statusHistory` and `currentLocation`, because both are always read together with the parcel and are bounded in size. Enumerated fields (`status`, `parcelSize`, `parcelType`, `paymentType`) are validated by the schema, and the same constants are reused by the controllers. Indexes exist on `trackingCode` (unique), `customer`, `agent` and `status`, matching the main query patterns.
- **Session** stores one document per refresh token: only its SHA-256 hash, the rotation `family`, expiry and revocation timestamps, and client metadata. A TTL index on `expiresAt` makes MongoDB delete expired sessions automatically [38].

Embedding the status history rather than using a separate events collection keeps the life-cycle update atomic (a single document write). This differs from earlier draft documents, which proposed separate tracking-event, notification, activity-log and report collections. Those collections were never implemented.

3.9 Parcel Lifecycle Model

The lifecycle is the core business rule of the system. Figure 3.5 shows the six states and the transitions permitted by the server.

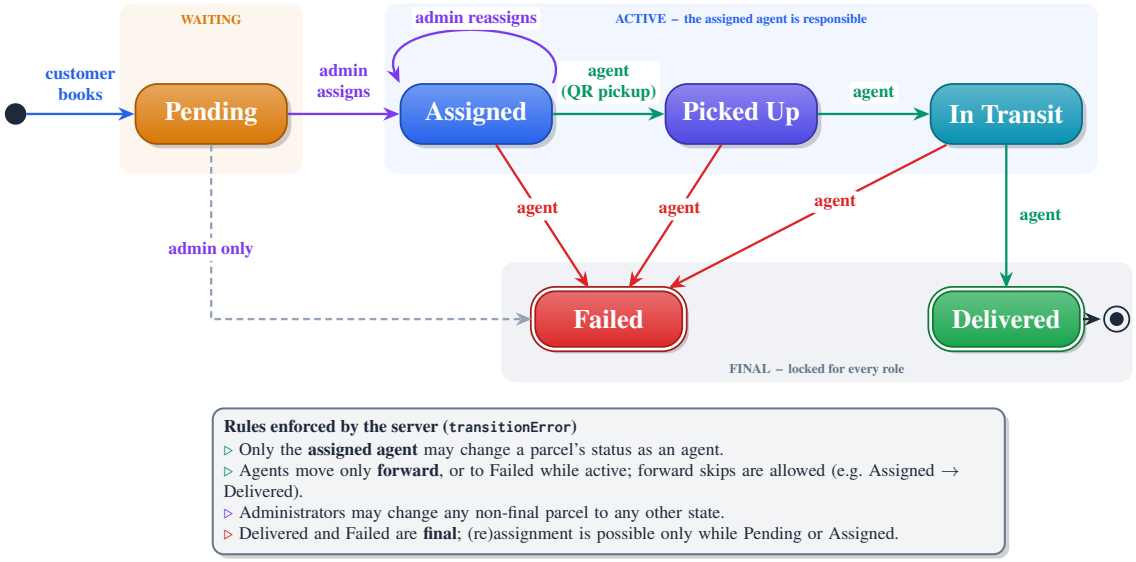


Figure 3.5: Parcel lifecycle state machine as enforced by the backend

Three design decisions deserve comment. First, *responsibility is explicit*: only the assigned agent may change a parcel's status as an agent, and active states require an assigned agent. Second, *final states are immutable*: once Delivered or Failed, a parcel cannot change, even by an administrator, which protects the reliability of records and analytics. Third, the rules for agents are *forward-only rather than strictly sequential*. An agent may skip intermediate states (for example from Assigned directly to Delivered). This keeps field use flexible but means that the history does not guarantee that every intermediate state was recorded. Test T56 documents this behaviour, and Chapter 6 recommends whether it should be tightened.

3.10 Interaction Design

Figure 3.6 traces the main workflow from booking to pickup across the three user interfaces, the API, the database and the Socket.IO rooms. Every state change follows the same pattern: authorise, validate, write to the database, then emit a scoped event and, where relevant, send an e-mail asynchronously.

Figure 3.7 shows the two paths by which location reaches watchers. The *socket path* is used for continuous agent tracking. The server verifies the sender and fans the position out to administrators and to watchers of that agent's active parcels. The *REST path* stores the position on In-Transit parcels so that it survives page reloads and is visible to public viewers.

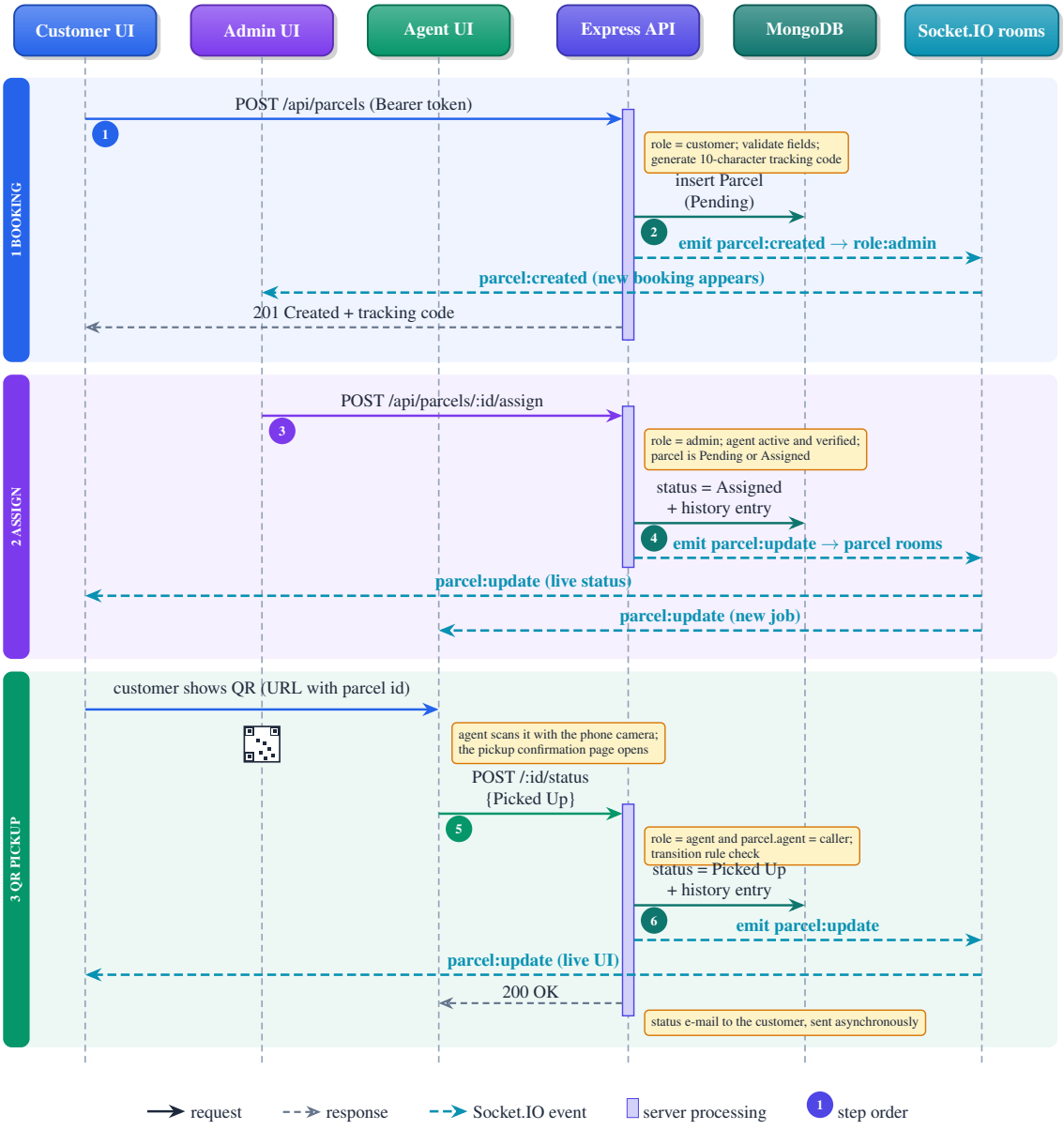


Figure 3.6: Sequence diagram: booking, assignment and QR-assisted pickup

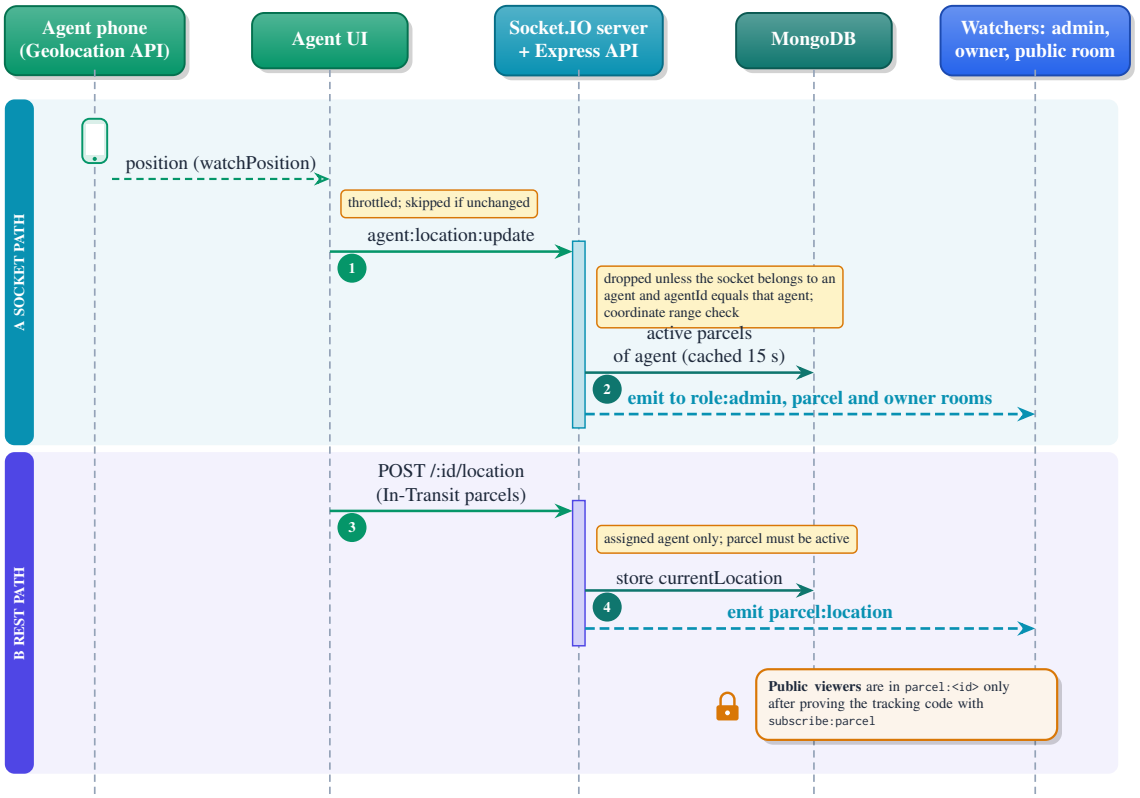


Figure 3.7: Sequence diagram: live agent location

3.11 API and Real-Time Design

REST API. The backend exposes 25 REST endpoints under `/api` plus a health check. Table 3.5 groups them.

Table 3.5: REST API groups

Base path	No.	Purpose and access
`/api/auth`	8	Register, verify e-mail, resend code, login, refresh, logout, forgot and reset password. Public, individually rate-limited; refresh and logout require the custom CSRF header
`/api/parcels`	8	Public tracking by code (rate-limited); authenticated list, create (customer), read (owner, assigned agent or admin), assign (admin), status (admin or agent), location (agent) and delete (admin)
`/api/users`	5	Own profile; list users, create agent, resend invitation, enable or disable (admin)
`/api/assignments`	1	Active, activated agents available for assignment (admin)
`/api/analytics`	3	Dashboard metrics, CSV export, PDF export (admin)
`/api/geocode`	1	Rate-limited proxy to Photon with Nominatim fallback, biased to Bangladesh

Real-time events. Table 3.6 lists the Socket.IO events.

Table 3.6: Socket.IO events

Event	Direction	Meaning and scope
`subscribe:parcel` / `unsubscribe:parcel`	client → server	Join or leave `parcel:<id>`. Joining requires ownership, assignment, admin role, or the correct tracking code (at most 25 parcel rooms per socket)
`agent:location:update`	both	Agent publishes own position. Server forwards to `role:admin` and to watchers of the agent's active parcels
`request:agent:location`	both	Admin asks for an agent's position. Server replies with the last known position and forwards the request to that agent only
`parcel:created`	server → client	New booking, sent to administrators
`parcel:update`	server → client	Status or assignment change, sent to parcel watchers
`parcel:location`	server → client	Stored location change, sent to parcel watchers

Room scoping. Figure 3.8 shows how events are scoped. Every authenticated socket joins `user:<id>`, and administrators also join `role:admin`. Parcel rooms are joined on request after an authorisation check. Anonymous sockets can join a parcel room only by presenting the parcel's tracking code, which mirrors the public REST endpoint. No event is broadcast to all sockets. An earlier version of the system broadcast agent locations globally. This design replaced it during the security hardening.

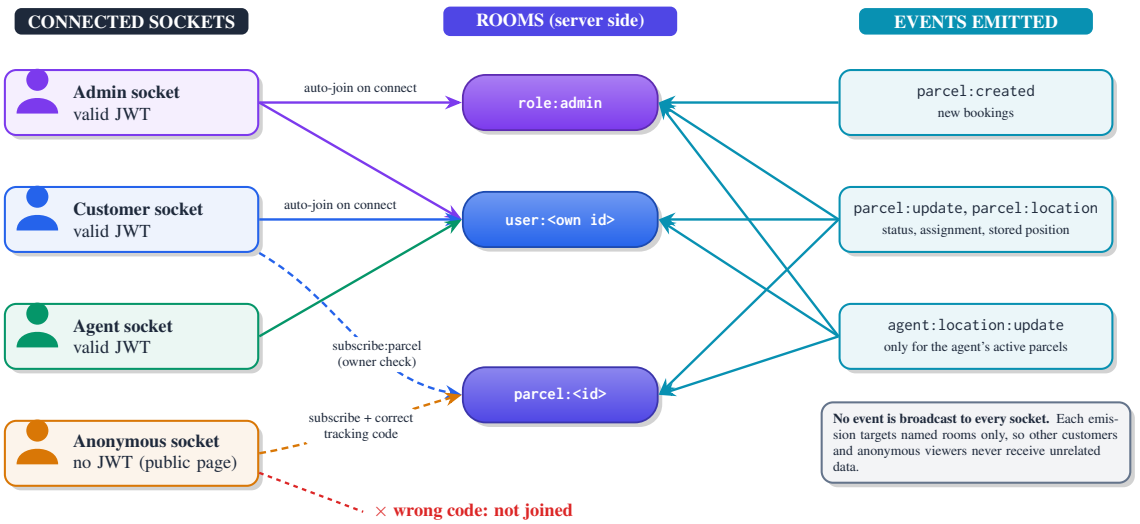


Figure 3.8: Scoping of real-time events through Socket.IO rooms

3.12 Security and Access-Control Design

The security design was refined in two review rounds recorded in the project's security review. Each finding was fixed in the code, and the fixes are present in the current source. Table 3.7 maps the main threats to controls and to the OWASP Top 10 categories [23].

Table 3.7: Threats, controls and OWASP Top 10 (2021) categories

Threat	Control in SCPMS	OWASP
Privilege escalation at sign-up	Role forced to `customer`; agents only created by admins	A01
Reading or changing others' parcels (IDOR)	Ownership/assignment checks in controllers; role guards on routes	A01
Token theft via script injection	Access token kept only in memory (15 min); refresh token in `HttpOnly`, `SameSite=Lax` cookie, path `/api/auth`	A07
Stolen refresh token	Rotation on every use; reuse after 30 s grace revokes the token family	A07
Revocation after password reset or disabling	`tokenVersion` embedded in tokens and checked on every request and socket connection	A07
CSRF on cookie endpoints	`SameSite=Lax` plus mandatory `X-Requested-With` header [27]	A01
Password guessing	bcrypt (cost 12) [29]; lockout after 5 failures; per-endpoint rate limits	A07
Account enumeration	Identical responses for unknown e-mails; dummy bcrypt comparison for timing	A07
NoSQL injection	Keys starting with `$` or containing `.` removed from body, query and params; type checks on ids [24]	A03
Location spoofing	Socket accepts an agent's position only from that authenticated agent	A01
Data leakage on public pages	Reduced public view; room-scoped events	A01
Misconfiguration	Production refuses to start without a JWT secret of at least 32 characters or an origin allow-list; `helmet` headers [39]; generic error responses	A05
Vulnerable dependencies	Audit fixes applied; production dependency audit clean (Section 4.5)	A06

The password policy requires 8–64 characters including at least one letter and one digit. It meets the NIST minimum length, and the lockout and rate limits satisfy the NIST rate-limiting requirement. However, the composition rule is something NIST recommends against, and new passwords are not screened against breached-password lists [30]. These two deviations are recorded as limitations.

3.13 Technology Selection and Justification

Table 3.8 summarises the main technologies and the reason for each choice.

Table 3.8: Technology choices and justification

Technology	Justification
Node.js + Express 4	One language (JavaScript) across client and server; event-driven I/O suits many concurrent socket connections [20]; minimal framework with mature middleware [40]
MongoDB + Mongoose 8	Document model fits a parcel with embedded history and location; schema validation and enums in Mongoose [41]; TTL indexes expire sessions [38]
React 19 + Vite 7	Component model for three role-specific interfaces; fast development server with an API proxy [42, 43]
Socket.IO 4	WebSocket transport with reconnection and rooms, which implement the access-scoped event design [21, 11]
Leaflet + OpenStreetMap + OSRM	No licence fees; open data with local (Bangla) labels; road routing [31, 12, 13]
Photon / Nominatim (via backend proxy)	Free OSM-based geocoding. The proxy adds rate limiting and Bangladesh bias, and identifies the application as required by the Nominatim policy [32]
qrcode + ZXing / BarcodeDetector	QR generation and scanning in the browser, with no native app or special hardware [33, 34]
jsonwebtoken, bcryptjs, helmet, express-rate-limit	Standard, well-documented building blocks for tokens [25], password hashing [29], headers [39] and throttling
Nodemailer + PDFKit	SMTP e-mail [44] and server-side PDF generation without external services
Tailwind CSS 3	Utility classes for a consistent responsive layout

3.14 Design Constraints and Assumptions

- **Single operator, single process.** The design assumes one courier organisation and one backend instance. Last-known agent positions are held in memory.
- **Browser-based devices.** Agents use a phone browser that supports geolocation and camera access over HTTPS. Location accuracy is that of the device.
- **Free third-party services.** Map tiles, routing and geocoding use public services with usage limits and no availability guarantee.
- **E-mail as the only notification channel.** Without SMTP configured in production, sign-up is disabled, because verification is mandatory.
- **Addresses as free text.** Addresses are geocoded on display rather than stored as coordinates, so ambiguous addresses may be placed inaccurately on the map.

- **Bangladeshi context.** Telephone validation accepts only Bangladeshi mobile numbers, and currency amounts are in Bangladeshi taka.

3.15 Development Environment and Tools

Table 3.9 lists the environment in which the system was built and evaluated.

Table 3.9: Development and evaluation environment

Item	Version / detail
Runtime	Node.js (evaluation run on v24.20.0; the Netlify build configuration pins Node 20)
Package management	npm workspaces (`backend`, `frontend`); `concurrently` for running both in development
Backend	Express 4.19, Mongoose 8.5, Socket.IO 4.7, jsonwebtoken 9, bcryptjs 2.4, helmet 8.3, express-rate-limit 8.6, Nodemailer 10.1, PDFKit 0.15 (minimum versions of the caret ranges in `package.json`); `nodemon` for development
Frontend	React 19.1, React Router 7.8, Vite 7.1, Tailwind CSS 3.4, axios, socket.io-client 4.8, Leaflet 1.9 with leaflet-routing-machine, qrcode, @zxing/browser, @react-google-maps/api
Database	MongoDB (deployment via `MONGODB_URI`; evaluation with MongoDB 7.0.14 through mongodb-memory-server)
Quality tools	ESLint 9 (frontend), npm dependency audit
Evaluation machine	Windows 11 Pro, AMD Ryzen 5 5600X (6 cores, 12 threads), 15.9 GB RAM

3.16 Project Structure

The repository is an npm workspace with two packages, `backend` and `frontend`, whose organisation is summarised in Table 3.10.

The backend comprises about 1,900 lines of JavaScript and the frontend about 9,200 lines (including about 740 lines of translations).

3.17 Backend Bootstrap and Request Pipeline

`backend/src/index.js` loads environment variables before any other module (some modules read configuration when imported), then creates one HTTP server shared by Express and Socket.IO. In production, startup fails unless `CLIENT_ORIGIN` is set, and `utils/jwt.js` fails unless `JWT_SECRET` has at least 32 characters. A missing configuration value therefore stops the server instead of silently weakening it.

Every request passes through the same middleware chain: `helmet()` security headers; CORS restricted to the configured origins with credentials; a JSON body parser limited to 100 kB; `cookie-parser`; `sanitizeInput` (Section 3.27); request logging; and a global API

Table 3.10: Organisation of the repository

Folder	Contents and responsibility
Backend package	
`src/index.js`, `src/config`	Server start-up, middleware chain and database connection
`src/models`	Data models for users, parcels and sessions
`src/routes`	URL groups for authentication, parcels, users, assignments, analytics and geocoding
`src/controllers`	Business rules for authentication, parcels and analytics
`src/middleware`	Authentication and role guards, rate limits, input sanitising
`src/services`	Session handling, real-time socket handling and room-scoped messaging
`src/utils`	Token, validation and e-mail helpers
`scripts`, `render.yaml`	Creation of the first administrator and agent; Render deployment settings
Frontend package	
`src/pages`, `src/components`	Twenty page components for customers, agents, administrators and public visitors, plus shared interface parts
`src/context`, API and socket clients	Sign-in state, language choice, server communication and live updates
`src/translations`	English and Bangla interface text
Hosting configuration	Vercel, Netlify and Vite settings for the `/api` proxy and security headers

rate limit of 600 requests per 15 minutes per client. Unknown `/api` paths return a JSON 404. A central error handler converts malformed JSON to 400 and oversized bodies to 413, and returns a generic 500 message for other errors, never a stack trace. Async route handlers are wrapped so that rejected promises reach this handler instead of leaving the request hanging.

3.18 Authentication and Session Management

Registration and verification. Only customers can register themselves (Figure 3.9). The controller normalises the name, e-mail and Bangladeshi mobile number (`01XXXXXXXXX`, optionally prefixed with `+88`). It checks the password policy (8–64 characters, letters and digits) and requires acceptance of the terms. The role is always set to `customer`, whatever the request contains. A six-digit code from `crypto.randomInt` is e-mailed. Only its SHA-256 hash is stored, with a 10-minute expiry, at most five attempts, a 60-second resend cooldown and a constant-time comparison. Without SMTP credentials in development, the mailer prints the message to the server log instead of sending it. This behaviour is used by the test harness.

Login. The login handler looks up the user with the hidden lockout fields. For unknown e-mails it still performs a bcrypt comparison against a dummy hash, so response time

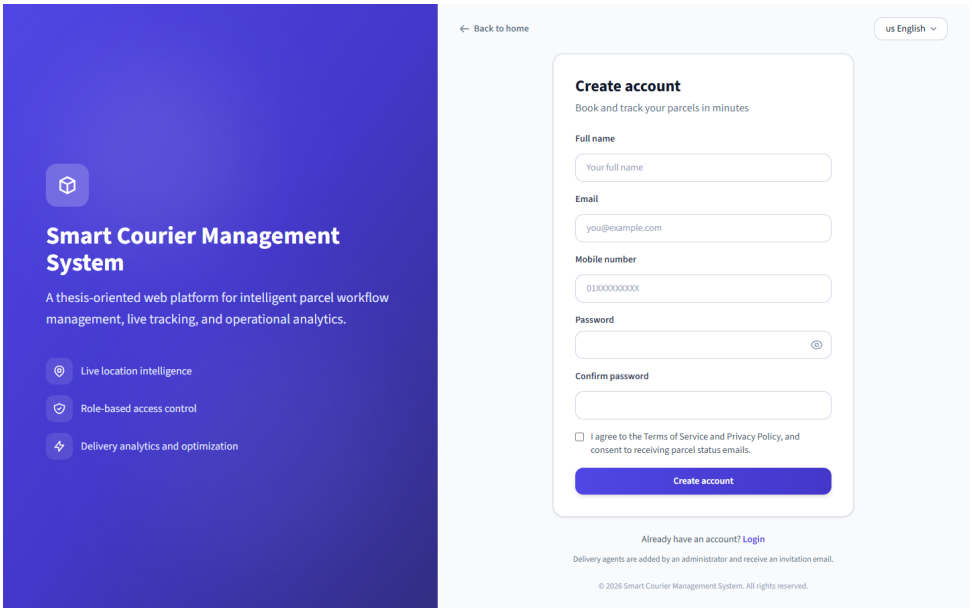


Figure 3.9: Customer registration page (running system)

does not reveal whether an account exists. The lock is checked *before* the password, so a correct guess during a lockout also fails. Five consecutive failures lock the account for 15 minutes. The account's active and verified states are revealed only after the password has been proven correct.

Tokens and sessions. A successful sign-in returns a JWT access token (HS256, issuer-checked, 15 minutes) containing the user id, role and `tokenVersion` (`tv`). It also sets a refresh cookie `scpms_rt`, a 384-bit random value, `HttpOnly`, `Secure` in production, `SameSite=Lax`, with path `/api/auth`. The frontend keeps the access token only in a JavaScript variable (`api.js`), never in `localStorage`. A timer refreshes the token one minute before expiry, and concurrent 401 responses share a single refresh request. Figure 3.10 shows the complete flow. When a refresh request arrives, the server handles the presented cookie as follows:

1. It looks up the session by the hash of the cookie value. An unknown, expired or revoked session is refused (`NO_SESSION`).

2. If the session was already rotated less than 30 seconds earlier, the request is treated as a harmless race between two browser tabs (`REFRESH_RACE`).

3. If the session was rotated longer ago, the old cookie has been replayed, which indicates theft. Every session in the same rotation family is revoked (`TOKEN_REUSE`), so the legitimate user must also sign in again.

4. Otherwise the session is marked as rotated, a new session with a fresh cookie is created in the same family, and a new access token is returned.

The 30-second grace window handles a real case found during development: two browser tabs refreshing at the same moment present the same cookie, and the second request should not be treated as theft. The client retries once after a short delay when it receives

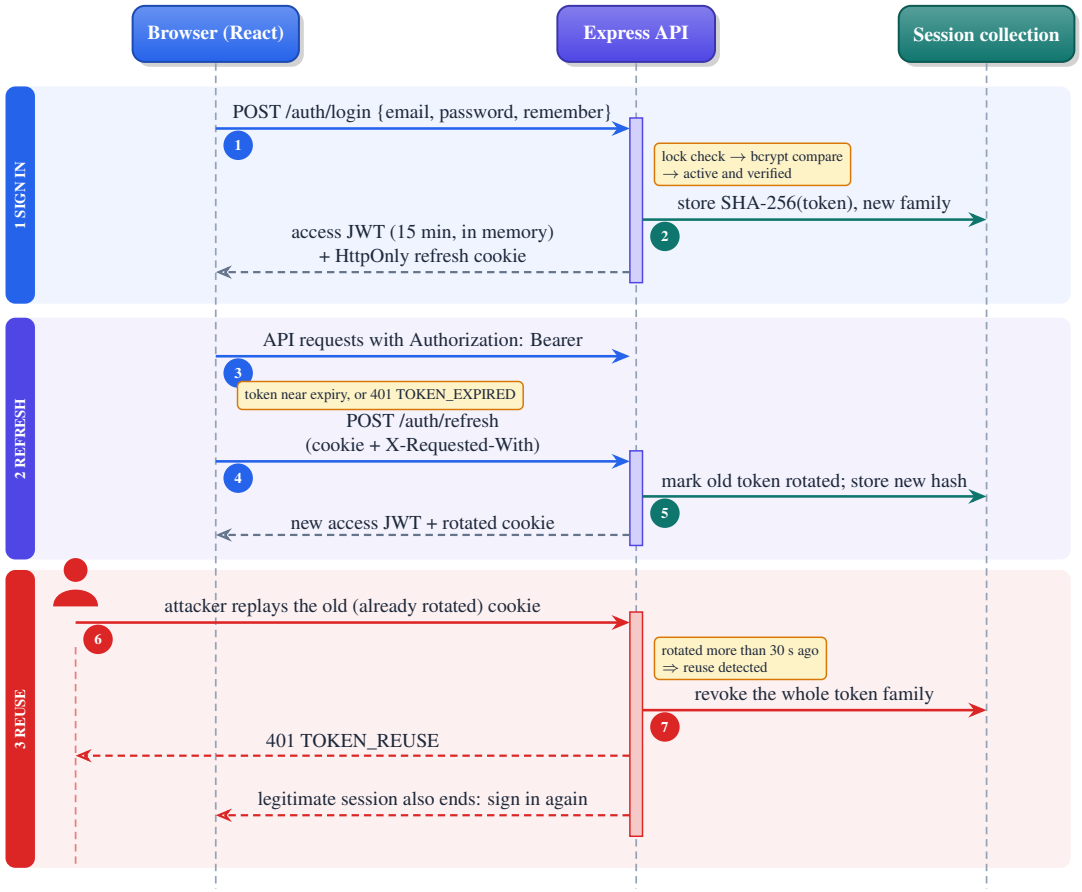


Figure 3.10: Sequence diagram: sign-in, silent refresh with rotation, and reuse detection

`REFRESH_RACE`. Sessions last one day, or 30 days with "Keep me signed in". The two cookie-authenticated endpoints (`/auth/refresh` and `/auth/logout`) additionally require the header `X-Requested-With: XMLHttpRequest`, which a cross-site form cannot send.

Password reset and revocation. The forgot-password endpoint always returns the same message. When the account exists, it e-mails a single-use link whose token is stored hashed and expires after 30 minutes. Completing a reset increments `tokenVersion` and revokes all sessions. The same mechanism signs a user out everywhere when an administrator disables the account. `requireAuth` reloads the user on every request and rejects a token whose `tv` no longer matches.

3.19 Authorisation

Authorisation is enforced on the server in two layers. The first layer is route guards: `requireAuth` verifies the token and loads the user, and `requireRoles(...)` admits only the listed roles. For example, booking is `requireRoles('customer')`, assignment and deletion are `requireRoles('admin')`, and location updates are `requireRoles('agent')`. The second layer is object-level checks in the controllers. `getParcel` returns a parcel only to its owner, its assigned agent or an administrator. `listParcels` adds a filter on `customer` or `agent` according to the role. `updateStatus` and `updateLocation` accept agents only for parcels assigned to them.

On the client, `ProtectedRoute`, `AdminRoute`, `DeliveryAgentRoute` and `CustomerRoute` redirect users away from pages for other roles. These guards improve usability but are not a security boundary, since the server repeats every check.

3.20 Parcel Booking and Lifecycle

Booking. Customers book from their dashboard (Figure 3.11). `createParcel` validates the addresses (1–300 characters), the enumerated fields, the COD amount (0–500,000, forced to 0 for prepaid parcels) and the notes ($\leq$500 characters). It then creates the parcel with an initial Pending history entry. The tracking code consists of ten characters drawn with `crypto.randomInt` from a 32-symbol alphabet that omits the easily confused characters 0, O, 1 and I. This gives $32^{10} = 2^{50}$ possible codes, which together with the rate limit on the public endpoint makes enumeration impractical. On the rare event of a duplicate code, creation is retried up to three times.

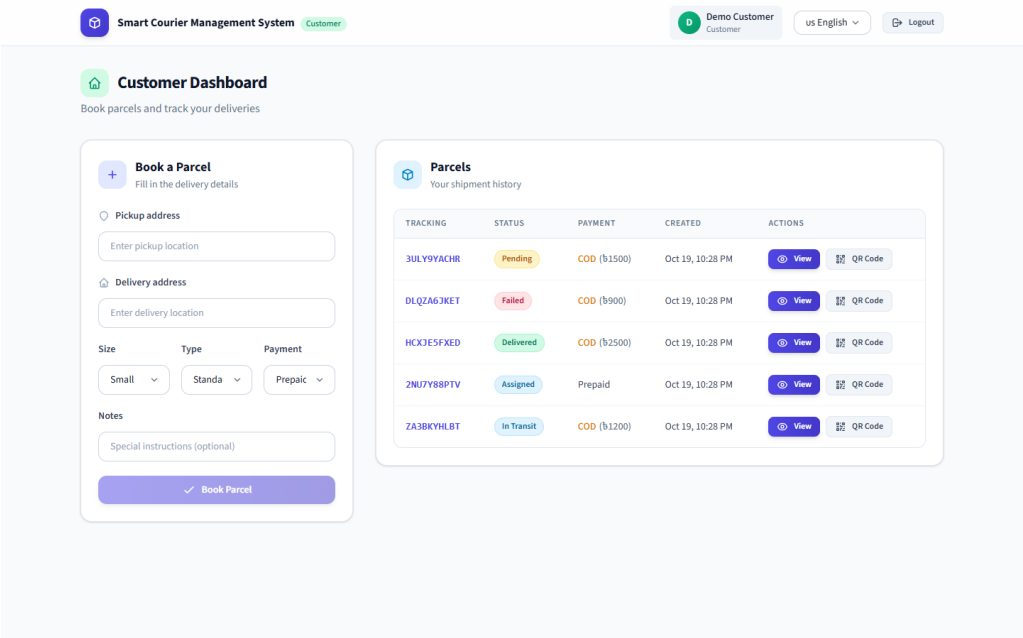


Figure 3.11: Customer dashboard with booking form and parcel list (running system, demonstration data)

Transitions. All status changes go through one server-side rule, which encodes the state machine of Figure 3.5. The rule applies its checks in a fixed order (Table 3.11). The first check that fails produces an explanatory error message, and the change is refused.

After a permitted change, the controller appends a history entry (status, time, actor and role) and saves the parcel. It then refreshes the agent's cached room list, pushes `parcel:update` to the watchers, and starts the status e-mail without waiting for it.

The agent's parcel page (Figure 3.12) shows the progress along the lifecycle, a status selector and controls for sending the current or a searched location.

Table 3.11: Order of checks applied to every requested status change

No.	Check	Outcome
1	The requested status equals the current status	Refused (“already in this state”)
2	The parcel is Delivered or Failed	Refused for every role (final states are locked)
3	The requester is an administrator	Allowed
4	The parcel is not yet assigned to an agent (Pending)	Refused for agents
5	The requested status is Failed	Allowed for the assigned agent
6	The requested status lies before the current one, or is Pending or Assigned	Refused (agents only move forward)
7	Any other forward move, including skipping states	Allowed

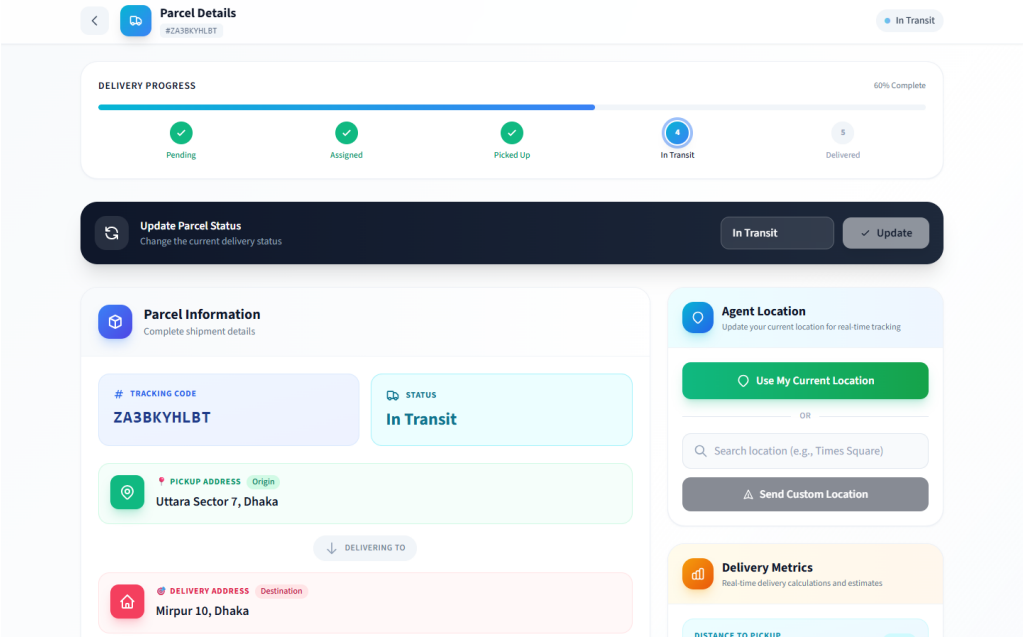


Figure 3.12: Agent parcel page with lifecycle progress, status update and location controls (running system)

3.21 Real-Time Layer

`services/socket.js` registers a Socket.IO middleware that reads `socket.handshake.auth.token`. If the token is valid and the user is active with a matching `tokenVersion`, it attaches `{id, role}` to the socket. Otherwise the socket stays anonymous. On connection, authenticated sockets join `user:<id>`, and administrators also join `role:admin`. The client reads the in-memory token at every (re)connection and reconnects after sign-in and sign-out, so the server always re-evaluates identity.

`services/realtime.js` provides `emitToParcel`, which sends to `role:admin`, `parcel:<id>`, the owning customer's room and the assigned agent's room, and `emitToAdmins`. Controllers emit only through these helpers. When an agent's browser sends a position, the server processes it in four steps:

1. It discards the message unless the socket belongs to a signed-in agent and the agent identifier in the message is that agent's own identifier. This prevents anyone from spoofing another agent's position.
2. It discards coordinates outside the valid latitude and longitude ranges.
3. It stores the position in memory as the agent's last known location, with a timestamp.
4. It forwards the position only to the administrators' room and to the rooms of the agent's active parcels (their watchers and owning customers).

To avoid one database query per GPS update, the list of rooms for an agent's active parcels is cached for 15 seconds and refreshed whenever an assignment or status changes.

Administrators follow agents on the tracking page (Figure 3.13). Selecting an agent loads that agent's parcels and sends `request:agent:location`. The server replies with the last known position and asks that agent's client, and only that client, for a fresh one.

3.22 Location, Maps and Estimated Arrival

Capturing location. The agent dashboard calls `navigator.geolocation.watchPosition` with high accuracy. Each new position is throttled and skipped if unchanged, emitted on the socket, and posted through REST to every parcel that is In Transit. The agent's parcel page additionally sends the current position every 20 seconds, and allows a typed address to be geocoded and sent instead (for example when GPS is unavailable). The server accepts REST location updates only from the assigned agent and only for active parcels.

Geocoding. Addresses are free text. The backend route `/api/geocode/search` appends "Bangladesh", queries Photon biased towards Dhaka, and falls back to Nominatim with `countrycodes=bd` and an identifying `User-Agent` header. The route is rate-limited and caps the number of results at five, so it cannot be used for bulk lookups.

Maps and routes. Leaflet renders OSM tiles. leaflet-routing-machine requests a road route between the geocoded pickup and delivery points from the public OSRM demon-

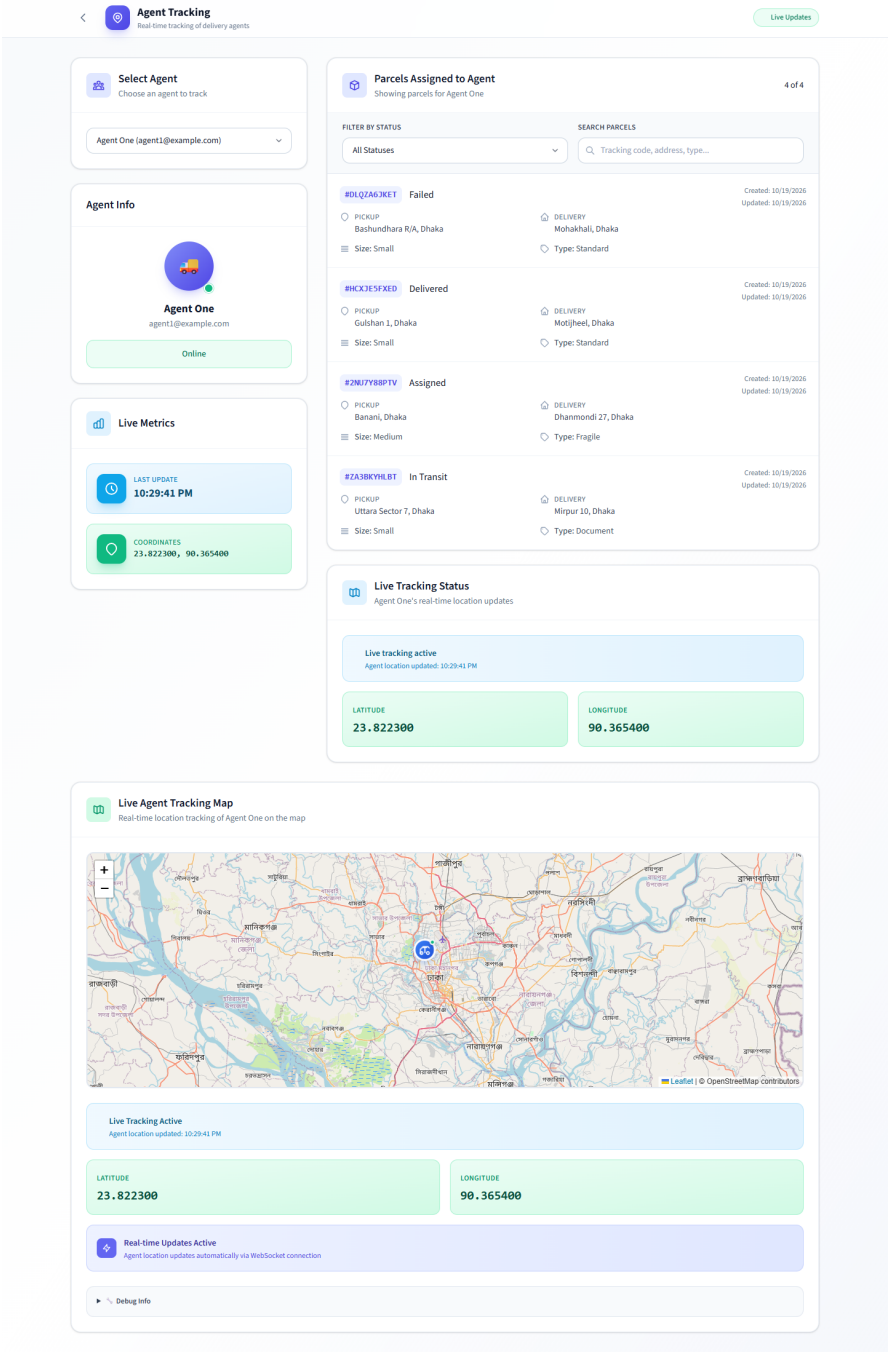


Figure 3.13: Administrator agent-tracking page receiving a live position from a connected agent (running system; position simulated)

stration server and draws it (Figures 3.14 and 3.15). One page (`PublicTrack.jsx`, at `/track/:code`) uses the Google Maps JavaScript API instead. Its API key is not present in the repository's production environment file, so that page shows a map error unless the key is configured on the hosting platform. The Leaflet page `/track-parcel`, linked from status e-mails, does not depend on Google.

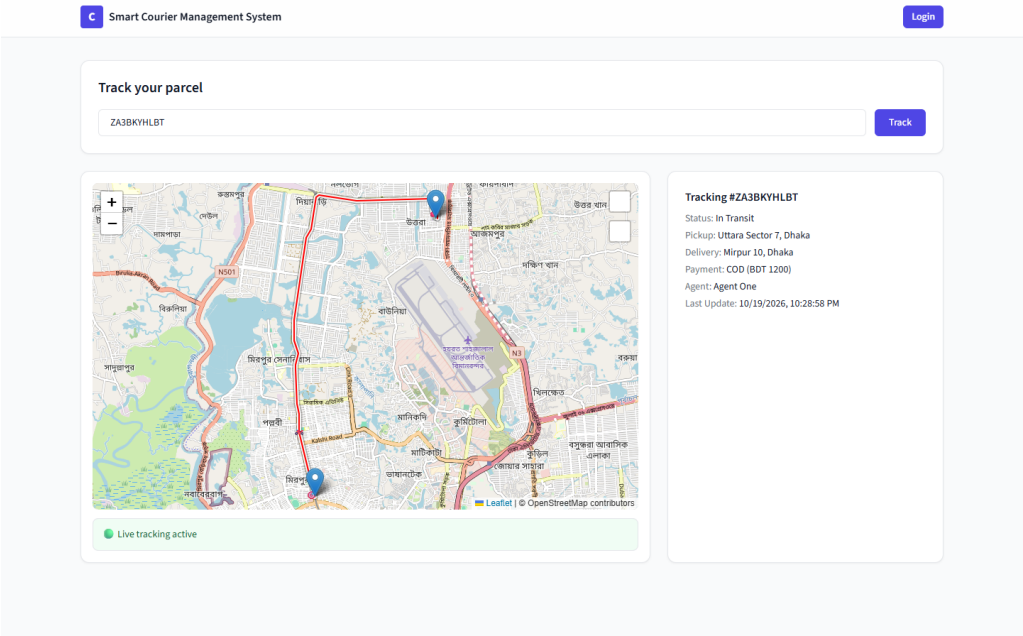


Figure 3.14: Public tracking by code with OSM map and OSRM route; no customer data is shown (running system)

Estimated arrival. The customer parcel page computes the straight-line (haversine) distance from the agent's latest position to the geocoded delivery point. It divides this distance by the agent's observed speed, or by 25 km/h when no speed is known, to display an ETA (Figure 3.15). This is a simple heuristic, not a road-network or traffic-aware prediction, and it is presented as such in this report. The server stores `etaMinutes` only when a client supplies it.

3.23 QR-Assisted Pickup

For each parcel the customer can open a QR page (Figure 3.16a). The page encodes the URL `<origin>/customer/parcel/qr/<parcelId>` with the `qrcode` library at error-correction level M, and offers a print button. The agent opens "Pickup Scan" from the dashboard. The scanner uses the browser's native `BarcodeDetector` when available, sampling the rear camera every 250 ms, and otherwise falls back to ZXing. It extracts the parcel id from the scanned URL and opens the confirmation page (Figure 3.16b). Confirming sends `{status: "Picked Up"}`. If the camera cannot be used, the agent can paste the link or the parcel id.

It is important to state precisely what the QR step achieves. It *identifies* the parcel quickly and reduces the chance that an agent confirms the wrong parcel. It is *not* a cryptographic proof of handover. The QR content is the parcel's database identifier, and the manual

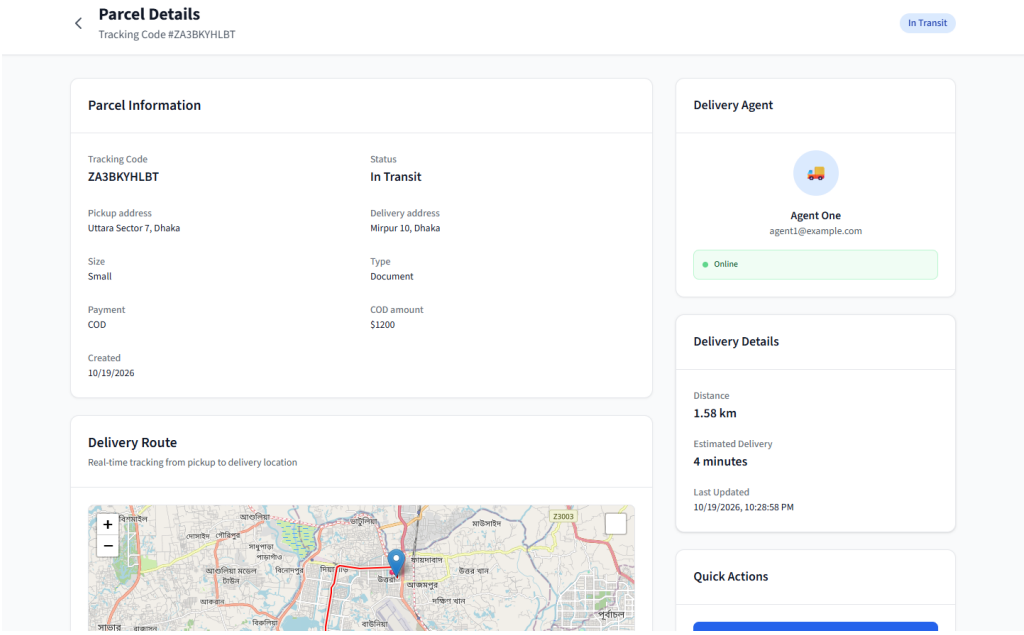
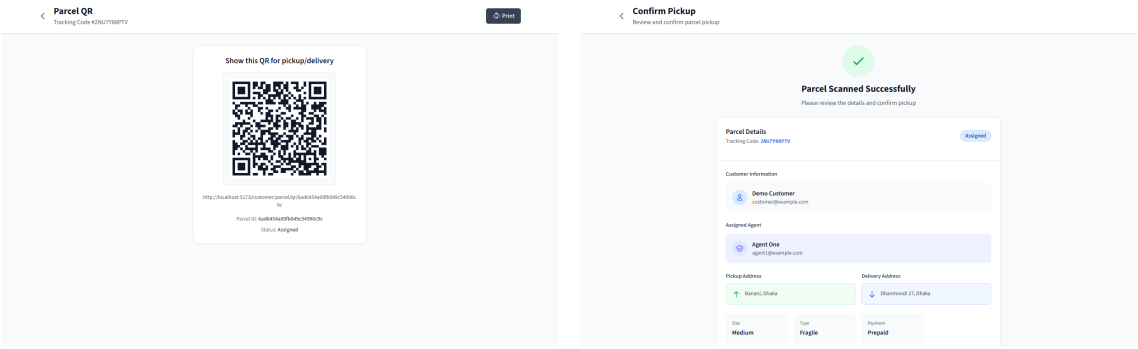


Figure 3.15: Customer parcel page with agent card, distance and heuristic ETA (running system). The COD amount is shown with a "$" sign, a currency-symbol defect recorded in Chapter 4



(a) Customer's parcel QR page **(b)** Agent's pickup confirmation page

Figure 3.16: QR-assisted pickup (running system). The confirmation page is normally reached by scanning; for the screenshot it was opened directly by URL

fallback accepts the same identifier. The security guarantee comes from the server, which accepts the Picked Up transition only from the agent assigned to that parcel. The QR page's heading mentions "pickup/delivery", but scanning is implemented only for pickup. Earlier project documents that describe QR verification at delivery, or OTP delivery confirmation, describe features that are not present: the OTP interface was removed during the security review because it had no backend.

3.24 Notifications

`utils/mailer.js` builds HTML e-mails with a shared layout and escapes all user-supplied text to prevent HTML injection. Four message types exist: the verification code, the password-reset link, the agent invitation (72-hour link) and the status-change notification. The status notification links to the public tracking page. It is sent asynchronously after the database update, and failures are only logged, so an unavailable SMTP server cannot delay or fail a status change. In production, an unconfigured mailer makes the verification, reset and invitation operations return 503. If an invitation cannot be e-mailed, the administrator is shown the link to share by another channel.

3.25 Analytics, Export and User Management

The administrator dashboard (Figure 3.17) shows three indicators computed by `dashboardMetrics`: bookings created since local midnight, the number of parcels in the Failed state, and the sum of COD amounts of delivered COD parcels (a MongoDB aggregation). The indicators refresh when `parcel:created` or `parcel:update` events arrive. The bookings table supports text search by tracking code or status and per-row assignment, and the assignment control locks once a parcel is past Assigned. The CSV export streams one row per parcel (tracking code, status, payment type, COD amount, creation time) with proper quoting. The PDF export writes the same fields as a simple list with PDFKit. The users panel lists all accounts with their state (active, unverified or invitation pending, disabled) and lets the administrator add agents, resend invitations and disable or re-enable accounts.

The analytics are deliberately basic. The system does not compute success rates over time, revenue trends, monthly reports, heat maps or delay risk, although some early documents listed them. They remain future work.

3.26 Frontend Implementation and Internationalisation

The frontend is a React single-page application. `App.jsx` defines public routes (landing, tracking, login, registration, verification, password reset), guest-only routes and role-guarded route groups. `AuthContext` restores the session on page load by calling the refresh endpoint (the access token is not persisted), schedules the next refresh and reconnects the shared socket when identity changes. `api.js` attaches the bearer token, retries once after a successful refresh on token errors, and converts server errors into an `ApiError` carrying the machine-readable code (for example `EMAIL_NOT_VERIFIED`). The UI uses these codes to show helpful messages.

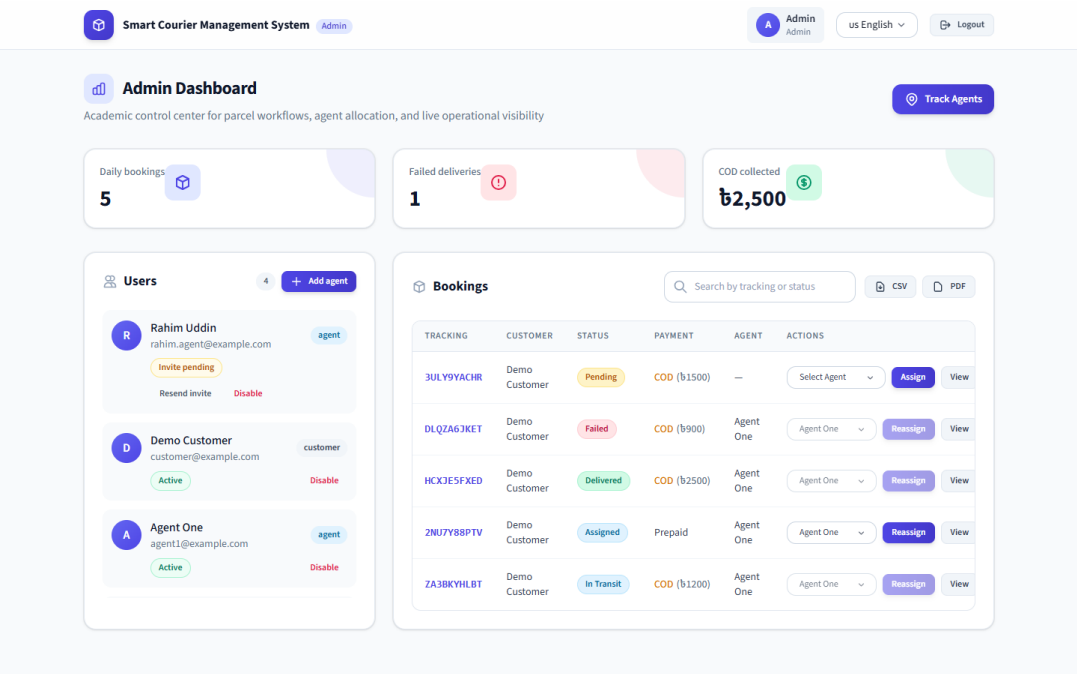
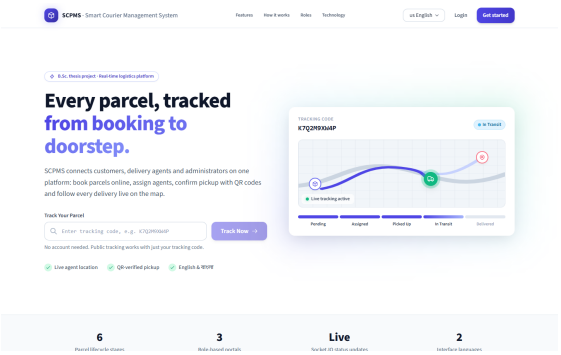
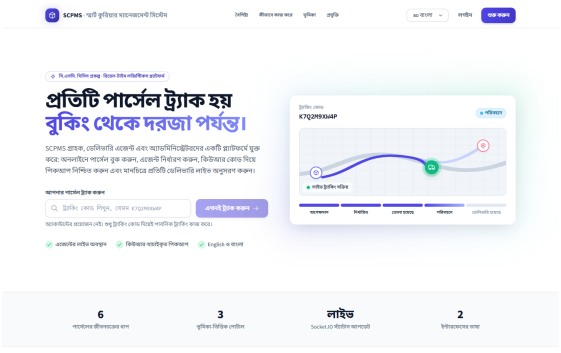


Figure 3.17: Administrator dashboard: indicators, user management, bookings with assignment and CSV/PDF export (running system, demonstration data)



(a) English



(b) Bangla

Figure 3.18: Landing page with public tracking entry in both interface languages (running system)

`LanguageContext` stores the chosen language in `localStorage`, and each component reads its strings from `translations/en.js` or `bn.js`, which hold about 320 entries each. Figure 3.18 shows the landing page in both languages. The agent dashboard (Figure 3.19) summarises the agent's parcels, shows the socket connection state and a live OSM map of the agent's own position, and links to the pickup scanner.

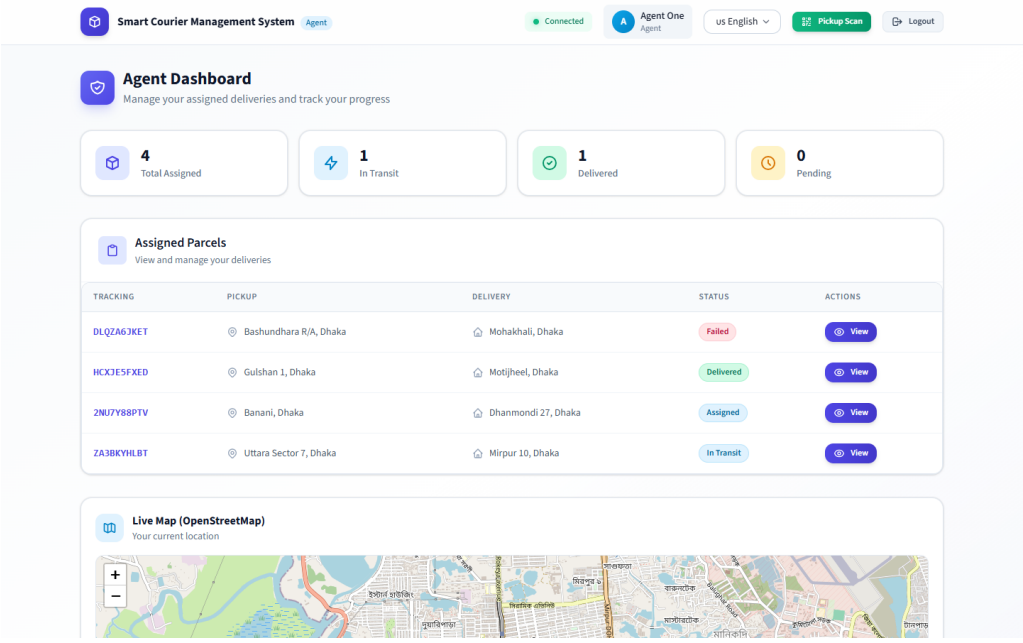


Figure 3.19: Delivery-agent dashboard (running system, demonstration data)

3.27 Validation, Error Handling and Hardening

Input is validated at three levels. First, `middleware/sanitize.js` recursively removes every key that begins with `$` or contains `.` from the body, query string and route parameters, so values such as `{"$ne": null}` can never reach MongoDB as operators [24]. Second, controllers check types and ranges explicitly: identifiers with `isObjectId`, coordinates with `isValidLatLng`, strings with length limits, and enumerations against the shared constants. Third, Mongoose schema constraints act as a final safeguard. Mongoose's `strictQuery` option discards unknown query fields.

Rate limits are applied per endpoint: login (20 per 15 minutes), registration (10 per hour), code verification and password reset (20 per 15 minutes), e-mail-sending endpoints (5 per 15 minutes), refresh (120), public tracking (120) and geocoding (300 per 15 minutes), on top of the global limit. The number of trusted proxy hops is configurable (`TRUST_PROXY_HOPS`), so that the limiter sees real client addresses behind the hosting proxies. The frontend hosting configuration adds `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, HSTS and a `Permissions-Policy` that allows camera and geolocation only for the site itself.

3.28 Deployment and Execution

Figure 3.20 shows the deployment that the repository is configured for. `backend/render.yaml` defines a Render web service with a `/health` check, `NODE_ENV=production`, two trusted proxy hops, a generated `JWT_SECRET`, and placeholders for the database URI, allowed origin, application URL and SMTP credentials. `frontend/vercel.json` and `netlify.toml` serve the built SPA, rewrite `/api/*` to the Render service and set security headers. `frontend/.env.production` sets `VITE_API_BASE=/api` and the socket URL. Whether this deployment is currently running was not verified as part of this work.

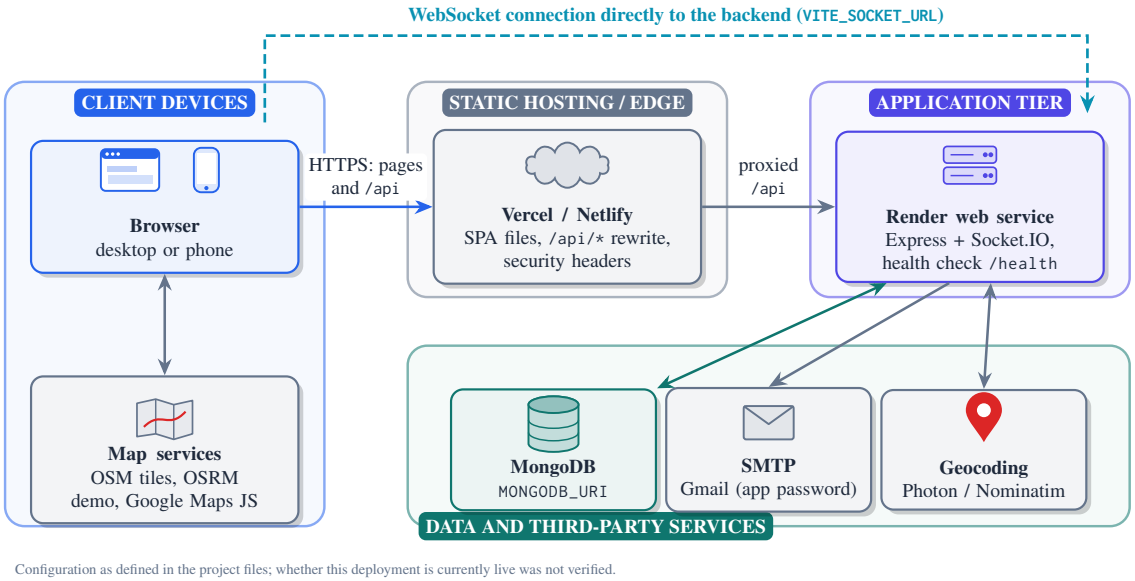


Figure 3.20: Deployment view as configured in the repository

For local execution, the dependencies of both packages are installed from the repository root. A seed script creates the first administrator and agent; in production, their passwords must be supplied through environment variables. A single development script then starts the API on port 5000 and the web client on port 5173.

3.29 Implementation Challenges

- **Securing real-time channels.** The first version trusted the `agentId` sent by any socket and broadcast all agent positions to every client, so locations could be spoofed and leaked. Authenticating sockets with the access token and scoping every event to rooms solved both problems. It required the room cache to keep database load low during continuous GPS updates.
- **Cookies across hosting providers.** With the frontend and API on different domains, browsers that block third-party cookies dropped the refresh cookie. Routing `/api` through the frontend host made the cookie first-party.
- **Configuration loaded too late.** Some modules read `process.env` when imported, before `.env` had been loaded, so local SMTP and JWT settings were silently ignored. Loading `dotenv` as the first import fixed this.

- **Concurrent refreshes.** Rotating refresh tokens broke sessions when two tabs refreshed together. The 30-second grace window and a single client retry handle this race without weakening reuse detection.
- **Free map services.** Geocoding of informal Bangladeshi addresses is sometimes imprecise, and public routing and geocoding services impose rate limits. A backend proxy with Bangladesh bias and a Photon-to-Nominatim fallback mitigates, but does not remove, this dependency.
- **Camera access.** Browser camera access requires HTTPS and user permission, and `BarcodeDetector` is not available everywhere. The ZXing fallback and manual entry keep pickup possible.

3.30 Summary

This chapter turned the problem and the literature into 21 functional and seven non-functional requirements, presented the design that meets them and described its implementation. The design consists of a layered single-process backend with a three-collection data model, a server-enforced six-state lifecycle with an embedded audit trail, room-scoped real-time events, and a layered security design mapped to the OWASP Top 10. The implementation realises this design in about 11,000 lines of JavaScript. The backend enforces authentication, authorisation and the parcel lifecycle in one place and delivers real-time events only to authorised rooms. The frontend provides three role-specific interfaces, public tracking, maps, QR-assisted pickup and a bilingual interface. The screenshots confirm that these features run end to end. Chapter 4 evaluates the implementation against the requirements.

Chapter 4

Analysis, Results and Discussion

4.1 Overview

This chapter evaluates the implemented system against the requirements of Chapter 3 and answers engineering question EQ3. It first states the evaluation objectives, environment and strategy (Sections 4.2–4.4). It then reports static analysis and build results (Section 4.5), executed functional, security and real-time tests (Sections 4.6–4.8), local performance observations (Section 4.9) and visual verification of the running system (Section 4.10). It closes with the defects found, requirements traceability and a discussion of the results. The evaluations that were not performed are listed in Chapter 5.

Throughout, the chapter distinguishes four kinds of evidence: (a) checks that were **executed** with recorded output; (b) behaviour verified by **inspection** of the code or of the running interface; (c) checks that were **not executed** but for which a procedure is proposed; and (d) earlier project documents whose claims could **not be verified**. The progress report and earlier slides contain figures such as a "98.6% booking success rate", a "0.96 test pass ratio", "180 ms average API response time" and "under 2 s location latency". Those documents label these figures as estimates for demonstration, so they are **not used** here.

4.2 Evaluation Objectives

E1. Verify that each functional requirement behaves as specified for valid input and rejects invalid input.

E2. Verify that access control holds for every role, including attempts to act on other users' data.

E3. Verify the session and account-security mechanisms (verification, lockout, rotation, reuse detection, revocation, CSRF defence, rate limits).

E4. Verify that real-time events reach authorised watchers and do not reach others.

E5. Assess code quality and dependency hygiene with standard tools.

E6. Obtain indicative, honestly qualified latency measurements on a local machine.

4.3 Test Environment

Table 4.1 describes the environment of the executed evaluation. The system under test was the unmodified backend source in the repository. The harness started it as a separate

process in a temporary working directory, so the developers' private `backend/.env` file (which may contain production credentials) was not loaded. The database was a throw-away in-memory MongoDB, and no production data or real e-mail was involved.

Table 4.1: Environment of the executed evaluation

Item	Detail
Date of run	19 October 2026
Hardware	AMD Ryzen 5 5600X (6 cores, 12 threads), 15.9 GB RAM
Operating system	Windows 11 Pro (build 26200), x64
Runtime	Node.js v24.20.0
Database	MongoDB 7.0.14 started by mongodb-memory-server (in-memory)
Backend configuration	`NODE_ENV=development`; random 64-character JWT secret; `EMAIL_VERIFICATION=required`; no SMTP credentials (e-mails printed to the log); `BCRYPT_ROUNDS=10` to shorten the run; seeded with `backend/scripts/seed.js`
Test clients	Node.js `fetch` with a cookie jar; socket.io-client 4.8.1 (WebSocket transport)
Static tools	ESLint 9.33 (frontend configuration), Vite 7.3 production build, npm dependency audit, Node.js syntax check
Visual verification	Headless Google Chrome via puppeteer-core 23.11, viewport 1440×900, fake camera device
Artefacts	`report/evidence/` (scripts, `results.json`, redacted server log, tool outputs, screenshots)

4.4 Testing Strategy

The repository contained no automated tests before this evaluation. A complete unit-test suite could not be added without restructuring the application code, which was outside the scope of documenting the system. The strategy therefore treats the backend as a black box and exercises it through its public interfaces, as a client would:

- **Static analysis and build** of the frontend, and syntax checks of all backend files, to assess code quality.
- **System-level black-box tests** of the REST API and Socket.IO events against the real backend and a real (in-memory) MongoDB. Each test case corresponds to one or more requirements and checks HTTP status codes, machine-readable error codes, response content and which sockets receive which events. Because the tests pass through the complete middleware chain, controllers, models and database, they act as integration tests as well as functional tests.
- **Negative and adversarial cases**: invalid input, other users' identifiers, forged and replayed tokens, injected query operators and spoofed socket messages.

- **Visual end-to-end verification**: scripted navigation of the running frontend in a browser, with screenshots (Chapter 3).
- **Local latency sampling** during the same run.

The harness consists of 66 test cases in `report/evidence/api-test/run-api-tests.cjs`. It creates its own data through the API (three customers, the seeded administrator and agent, and one invited agent), so each run starts from an empty database and is repeatable. A run takes under two minutes, including a deliberate 31-second wait that is needed to test token-reuse detection after the grace window.

4.5 Static Analysis and Build Results

Frontend build. The Vite production build completed successfully and transformed 448 modules. It produced a 1,879 kB JavaScript bundle (428 kB gzip-compressed) and a 108 kB stylesheet. Vite warned that the main chunk exceeds 500 kB, because the whole application, including map, QR and scanner libraries, ships as one bundle.

Frontend lint. ESLint with the project's own configuration reported 105 problems, 98 errors and 7 warnings (Table 4.2). None of them prevents the build. They do show maintenance debt and contradict an earlier internal note that the lint result was clean. The three `no-undef` errors concern `process.env.NODE_ENV`. Vite replaces this expression at build time, and the string is absent from the built bundle, so these three are false positives. The 30 duplicate keys in the translation files are genuine defects, since a later definition silently overrides an earlier one.

Table 4.2: ESLint findings for the frontend

Rule	Count	Interpretation
`no-unused-vars` (error)	45	Unused imports, state and helper functions in ten page components
`no-dupe-keys` (error)	30	15 duplicate keys in each of `en.js` and `bn.js`
`no-empty` (error)	18	Empty `catch` blocks that silently ignore errors
`no-undef` (error)	3	`process.env.NODE_ENV`; false positives (replaced by Vite)
`react-refresh/only-export-components` (error)	2	Context files export hooks and components together (development hot-reload only)
`react-hooks/exhaustive-deps` (warning)	7	Effects with incomplete dependency lists; possible stale values
Total	**105**	98 errors, 7 warnings

Backend. The Node.js syntax check reported no syntax errors in any of the 26 backend files. No backend lint configuration exists.

Dependencies. The npm audit of production dependencies reported **0 vulnerabilities** in the dependencies shipped to production. The full audit, including development tools, reported 8 advisories (6 high, 2 moderate) in the Tailwind CSS 3 build chain (`braces`, `po`

`stcss-selector-parser`). These affect only the build machine, and fixing them requires a major upgrade to Tailwind CSS 4.

4.6 Functional and Integration Test Results

All 66 executed test cases passed (Table 4.3). The paragraphs below describe the cases by area; each case is identified by its test ID.

Table 4.3: Summary of executed black-box tests by area

Area	Tests	Passed	Test IDs
System	3	3	T01, T03, T04
Robustness	1	1	T33
Auth	19	19	T05, T06, T07, T08, T09, T10, T11, T12, T13, T14, T15, T16, T17, T18, T19, T20, T21, T22, T38
Users	4	4	T24, T37, T61, T62
Access control	9	9	T23, T25, T30, T31, T32, T39, T41, T50, T58
Booking	4	4	T26, T27, T28, T29
Parcels	1	1	T64
Tracking	4	4	T34, T35, T48, T49
Assignment	3	3	T36, T42, T46
Lifecycle	9	9	T40, T44, T45, T47, T52, T53, T54, T55, T56
Real-time	2	2	T43, T51
Analytics	1	1	T57
Reports	2	2	T59, T60
Security	4	4	T02, T63, T65, T66
Total	**66**	**66**	

Accounts and authentication (FR-01–FR-03, FR-05, FR-17). Registration rejected a weak password, an invalid mobile number and missing terms acceptance with distinct error codes (T05–T07). A registration that tried to set `role: "admin"` produced a customer account that required verification (T08, T11). An unverified account could not sign in (T09), a wrong code was rejected (T10), and the correct code signed the user in with a correctly scoped `HttpOnly` cookie (T11). Duplicate e-mails were refused (T12). An administrator-created agent could not be assigned work until it had set a password through the invitation link (T24, T36–T38). Password reset is exercised through the same endpoint by T37, and the forgot-password response was identical for existing and unknown accounts (T65).

Parcels and lifecycle (FR-06–FR-11, FR-13, FR-16). Invalid sizes, excessive COD amounts and missing addresses were rejected (T26–T28). A valid booking produced a Pending parcel with a ten-character code drawn from the documented alphabet (T29). Assignment, pickup, transit and delivery followed the state machine (T42, T44, T47, T52). Backward moves, reassignment after pickup and any change after delivery were

refused, even for an administrator (T45, T46, T53). The history recorded the full sequence with the actor's role (T54), and an active parcel could be marked Failed (T55). T56 confirmed that an agent can move a parcel directly from Assigned to Delivered. This is consistent with the forward-only rule but is reported as a design finding (Section 4.11).

Analytics and export (FR-14, FR-15). After the scripted scenario, the dashboard returned exactly the expected values: 3 bookings today, 1 failed delivery, and 2,000 taka of COD collected from the two delivered COD parcels (T57). The CSV export contained a header and one row per parcel, and the PDF export was a valid PDF document (T59, T60).

4.7 Security and Access-Control Test Results

All test cases that target access control and security passed. In summary:

- **Role checks.** Customers could not list users, create agents, assign agents, change statuses or read analytics, and administrators could not book parcels (T23, T25, T30, T39, T41, T58).
- **Ownership checks.** A customer could not read another customer's parcel by its identifier (IDOR, T32) and saw only their own parcels (T31). Agents could not update or locate parcels that were not assigned to them (T40, T50).
- **Tokens and sessions.** Missing and forged tokens were rejected (T16, T17). The refresh endpoint refused requests without the CSRF header (T18), rotated the cookie (T19), tolerated a concurrent reuse within the grace window (T20) and, after the window, revoked the whole token family, including the legitimate holder's session (T21). Logout ended the session (T22). Disabling an account invalidated an access token that was still within its lifetime and blocked sign-in (T62).
- **Brute force and enumeration.** Five failed attempts locked the account, and a correct password during the lock was refused (T14). Wrong-password and unknown-account responses were identical (T13), and the sixth e-mail-sending request in the window was rate-limited with HTTP 429 (T66).
- **Injection and robustness.** A login body containing `$ne` operators was neutralised and returned 400 without a session (T63). Malformed JSON, malformed identifiers and unknown routes produced 4xx responses (T03, T04, T33), and `helmet` headers were present (T02).
- **Privacy.** The public tracking response contained no customer object, no notes and no actor identifiers in the history (T34).

4.8 Real-Time Behaviour

Seven sockets were connected during the scenario: an administrator, the parcel owner, an unrelated customer, an anonymous viewer that proved the tracking code, an anonymous viewer that supplied a wrong code, the agent, and a customer socket used to attempt spoofing. When the parcel was assigned, `parcel:update` reached the administrator, the owner

and the code-holding viewer, and did *not* reach the unrelated customer or the wrong-code viewer (T43). A stored location update reached the public viewer as `parcel:location` (T49). A forged `agent:location:update` sent from a customer socket on behalf of the agent was dropped. The agent's genuine update reached the administrator and the owner of the agent's active parcel, but not the unrelated customer (T51). These results confirm the room-scoping design of Figure 3.8.

4.9 Performance Observations

Latency was sampled during the same run (Table 4.4). Requests were sent *sequentially* from the same machine over the loop-back interface to the in-memory database.

Table 4.4: Locally measured latencies (sequential requests, loop-back interface, in-memory database)

Measurement (ms)	**n**	**Mean**	**Median**	**P95**	**Max**
HTTP round trip: `GET /health`	200	14.28	15.31	18.59	23.89
HTTP round trip: `GET /api/parcels/track/:code (public)`	100	14.72	15.3	22.46	26.79
HTTP round trip: `GET /api/parcels (admin, 2 parcels)`	100	14.36	15.29	17.08	20.34
Socket: agent emit → admin receives `agent:location:update`	50	0.32	0.29	0.46	1.15
Status request issued → owner socket has `parcel:update` (includes the HTTP round trip): 14.24, 15.23, 14.99 ms (3 samples)					

These figures need careful interpretation:

- The median HTTP round trip was about 15.3 ms for all three endpoints, including `/health`, which does no database work. An identical floor for a trivial and a non-trivial endpoint indicates that the value is dominated by client-side and operating-system overheads on this Windows machine (consistent with its timer granularity of about 15.6 ms), not by application processing. The values are therefore an *upper bound* for local processing time. They are not a measure of the server.
- The socket fan-out median of 0.29 ms shows that room-scoped delivery adds negligible overhead on one machine with a warm room cache.
- No network latency, concurrent load, realistic data volume or remote database was involved. The numbers **must not** be generalised to a deployed system, and they say nothing about scalability. A load test is proposed in Section 5.3.

4.10 Visual Verification of the Running System

The screenshots in Chapter 3 were produced by a script that started the backend and frontend, created demonstration data through the API and visited each page in headless Chrome. Inspecting them confirmed, by observation rather than assertion, that:

- the landing, registration and login pages render, and the interface switches completely to Bangla (Figure 3.18);
- the customer dashboard lists parcels in all six states with correct status badges and COD amounts (Figure 3.11);
- the QR page renders a scannable QR code encoding the parcel URL, and the pickup confirmation page shows the correct parcel and assigned agent (Figure 3.16);
- the public tracking page geocodes both addresses, draws an OSRM road route on OSM tiles and shows no customer data (Figure 3.14);
- the administrator dashboard shows indicators consistent with the demonstration data, invitation states and assignment locking (Figure 3.17);
- with an agent signed in on another tab, the administrator tracking page marks the agent online and displays the live (simulated) coordinates on the map (Figure 3.13).

Camera scanning itself was not verified with a physical QR code: headless Chrome supplies only a synthetic test pattern. E-mail rendering was not verified with a real SMTP server.

4.11 Defects and Findings

Table 4.5 lists the defects and design findings identified during the evaluation. Severity reflects the impact on the current single-operator use.

Table 4.5: Defects and findings

ID	Finding	Severity	Evidence and recommendation
F1	Agents may skip lifecycle states (e.g. Assigned → Delivered), so the history need not contain every step	Medium	T56; require consecutive transitions for agents, or record skipped states explicitly
F2	Google Maps API key absent from the production environment file; `/track/:code` shows a map error unless configured on the host	Medium	`frontend/.env.production`; configure the key or consolidate on the Leaflet tracking page

ID	Finding	Severity	Evidence and recommendation
F3	Routes and geocoding recomputed on location updates; dependence on rate-limited public OSRM and geocoding services	Medium	Code inspection; cache geocoded coordinates on the parcel and recompute routes only when endpoints change
F4	Debug button "Test agent location" visible to customers on the parcel page	Low	`CustomerParcelDetail.jsx`; remove or restrict to development builds
F5	COD amount shown with "$" instead of taka on the customer parcel page	Low	Figure 3.15; use the taka sign as elsewhere
F6	98 lint errors, including 30 duplicate translation keys and 18 empty `catch` blocks	Low	Table 4.2; fix and enforce lint in CI
F7	Two public tracking pages with different maps and live-update behaviour	Low	`PublicTrack.jsx` vs `TrackPublicParcel.jsx`; consolidate
F8	`/parcel/:id/route` is a public route but calls an authenticated endpoint	Low	`App.jsx`, `ParcelRoute.jsx`; guard the route or remove it
F9	QR page heading mentions "pickup/delivery" but scanning exists only at pickup	Low	`CustomerQrScanner.jsx`; correct the wording or implement delivery confirmation
F10	1.88 MB single JavaScript bundle	Low	Build output; split map and scanner code with dynamic imports
F11	Last-known agent positions kept in process memory; single-instance Socket.IO	Limitation	`services/socket.js`; persist positions and use a shared adapter to scale
F12	Password composition rule and no breached-password screening, contrary to NIST guidance	Low	`utils/validation.js`; adopt length-based policy with a blocklist [30]
F13	Development-only dependency advisories (8)	Low	npm audit; upgrade the Tailwind toolchain
F14	No automated test suite in the repository before this work; Postman collection outdated	Process	Adopt the harness in CI; regenerate the collection

4.12 Requirements Traceability

Table 4.6 links every requirement to its implementation and evidence. "Test" means verified by an executed test case. "Inspection" means verified by reading the code or observ-

ing the running interface, without an automated assertion.

Table 4.6: Requirements traceability matrix

Req.	Implementation	Evidence	Status
FR-01	`register`	T05–T08, T11, T12	Met (test)
FR-02	`login`, `verifyEmail`	T09–T11, T13–T15, T38	Met (test)
FR-03	`session.service`, `refresh`, `logout`	T16–T22	Met (test)
FR-04	`requireRoles`, ownership checks	T23, T25, T30–T32, T39–T41, T50, T58	Met (test)
FR-05	`user.routes.js`	T24, T36–T38, T61, T62	Met (test)
FR-06	`createParcel`	T26–T30	Met (test)
FR-07	`utils/tokens.js`	T29	Met (test)
FR-08	`listParcels`	T31	Met (test)
FR-09	`getParcelByTrackingCode`	T34, T35; Fig. 3.14	Met (test)
FR-10	`assignAgent`	T36, T42, T46	Met (test)
FR-11	`transitionError`, `updateStatus`	T40–T47, T52–T56	Met, with finding F1
FR-12	`updateLocation`, socket handlers	T48–T51; Fig. 3.13	Met (test)
FR-13	`statusHistory`	T54	Met (test)
FR-14	`dashboardMetrics`	T57	Met (test)
FR-15	`exportCsv`, `exportPdf`	T59, T60	Met (test)
FR-16	`deleteParcel`	T64	Met (test)
FR-17	`forgotPassword`, `resetPassword`	T65, T37 (shared endpoint)	Met (test; e-mail delivery not tested)
FR-18	QR and scanner pages	Fig. 3.16; T44 (server side)	Met at pickup (inspection); physical scan not tested
FR-19	`sendStatusEmail`	Code inspection	Implemented; not exercised (needs SMTP)
FR-20	`translations/`	Fig. 3.18	Met (inspection); duplicate keys (F6)
FR-21	Leaflet, OSRM, geocode proxy	Figs. 3.14, 3.15	Met (inspection); F2, F3

Req.	Implementation	Evidence	Status
Security	See Table 3.7	T02, T11–T22, T62–T63, T65–T66	Met for tested controls
Privacy	Public view, room scoping	T34, T43, T51	Met (test)
Responsive.	Socket.IO rooms	T43, T49, T51; Table 4.4	Met locally; not load-tested
Robustness	Error handler, validation	T03, T04, T33	Met (test)
Usability	Responsive UI, Bangla, fallbacks	Screenshots	Not formally evaluated
Maintain.	Layered modules	Code inspection; lint	Partly met (F6, F14)
Deploy.	Env configuration, health check	T01; configuration files	Configured; live deployment not verified

4.13 Discussion

The executed evidence supports a positive answer to EQ3 within clear limits. Every functional requirement that can be exercised through the API is met, and the security mechanisms behave as designed under adversarial input. The tests found no failures in authorisation, lifecycle enforcement or event scoping. In particular, behaviours that are easy to get wrong in web applications were verified rather than assumed: IDOR protection, revocation of still-valid tokens, refresh-token reuse detection and the exclusion of unauthorised sockets from parcel rooms.

The evaluation also shows where the system falls short. Code quality on the frontend is uneven (98 lint errors). Some user-interface defects remain (F2, F4, F5, F7–F9). One lifecycle rule (F1) is weaker than the word "sequential" would suggest. The QR step identifies parcels but does not prove a handover. Performance has been observed only locally, and the free map and geocoding services are an operational dependency (F3).

These results are consistent with the project's framing as an applied engineering project: the contribution is a working, secure and verifiable system, not a measured improvement over an existing operation. No claim of operational improvement (for example fewer delays or higher customer satisfaction) can be made without the field and user studies of Section 5.3.

4.14 Summary

Static analysis, a production build, a dependency audit and 66 executed black-box tests were used to evaluate SCPMS. All tests passed, covering accounts, sessions, access control, the parcel lifecycle, analytics, exports, injection resistance, rate limiting and real-time event scoping. The production dependencies have no known vulnerabilities. Local latency figures are reported with their limitations. Fourteen defects and findings were doc-

umented. The evaluations that could not be performed are listed in Chapter 5, together with procedures for carrying them out.

Chapter 5

Limitations

5.1 Overview

This chapter states the limitations of the work. Section 5.2 lists the limitations of the system and of its evaluation. Section 5.3 lists the evaluations that were not carried out, each with a procedure for future work. Identifiers such as F1 refer to the defects and findings in Table 4.5.

5.2 Limitations of the System and the Evaluation

- **Evaluation scope.** Tests were run locally against an in-memory database. There was no usability study, load test, field trial or verification of the live deployment, and latency figures are local upper bounds only.
- **Handover assurance.** QR codes identify parcels at pickup but contain only the database identifier and can be bypassed through manual entry. There is no confirmation step at delivery, so a delivery is recorded on the agent's word.
- **Lifecycle strictness.** Agents can skip intermediate states (F1).
- **Scalability and state.** The backend is a single process. Socket rooms and last-known agent positions live in memory, so they are lost on restart and not shared across instances.
- **External services.** Maps, routing and geocoding rely on free public services with usage limits. One tracking page depends on a Google Maps key that is not in the repository configuration.
- **Analytics.** Only three indicators and two simple exports exist.
- **Code quality.** The frontend has 98 lint errors and duplicated logic across map pages, and before this work the repository had no automated tests.
- **Records.** Some early project documents describe features that were never built.

5.3 Evaluations Not Performed

The following evaluations would strengthen the conclusions but were **not carried out**. No results are claimed for them. Each is listed with a reproducible procedure for future work.

- **Usability study.** No participants were recruited. Proposed procedure: 8–12 participants per role complete scripted tasks (book a parcel, confirm a pickup, assign an agent), measuring task completion, time on task and errors, followed by the System Usability Scale questionnaire.
- **Load and scalability test.** Proposed procedure: deploy the backend and a managed MongoDB in the target region; use a load generator (for example k6 or Artillery) to simulate increasing numbers of agents emitting a location every 5–10 s and customers polling or subscribing; record latency percentiles, error rates and CPU and memory use until saturation.
- **Field trial.** Agents' phones on real routes in Dhaka, measuring GPS accuracy, update frequency, battery impact and the time between a status change and its appearance on the customer's screen over mobile networks.
- **Cross-browser and device testing** of camera scanning (BarcodeDetector and ZXing paths) with printed QR codes on Android and iOS browsers.
- **E-mail delivery** through a real SMTP provider, including spam-folder placement.
- **Live-deployment verification** of the Render and Vercel/Netlify configuration.

5.4 Summary

The system was tested for functional correctness and security on a single machine, but not with real users, under load, in the field or as a live deployment. These limitations bound the conclusions drawn in Chapter 6, and the evaluations listed above are the natural next steps.

Chapter 6

Conclusion

6.1 Overview

This chapter summarises the project, assesses each objective against the evidence of Chapter 4, answers the engineering questions, and discusses contributions, lessons learned and future work. The limitations of the work are stated in Chapter 5.

6.2 Summary of the Project

The project set out to address the lack of parcel visibility, unclear responsibility, missing accountability and limited operational insight in manually coordinated courier operations. The resulting system, SCPMS, is a MERN web application with Socket.IO real-time messaging. Customers book and follow parcels, delivery agents confirm pickups with a QR scan and share their location, administrators assign work and monitor operations, and anyone holding a tracking code can follow a parcel without seeing personal data. The design centres on a server-enforced six-state lifecycle with an embedded audit trail, room-scoped real-time events and a layered security design. The implementation was evaluated with static analysis, a production build, a dependency audit, 66 executed black-box tests and screenshots of the running system.

6.3 Achievement of Objectives

Table 6.1 assesses the objectives of Table 1.1.

6.4 Answers to the Engineering Questions

EQ1: role-appropriate visibility of a shared lifecycle. A single parcel record, accessed through one API that applies role checks and ownership checks, gives each role a consistent but filtered view of the same lifecycle. Because all lifecycle rules live in one server function, every client sees the same states and transitions, and the embedded history provides a common account of what happened. Tests T31, T32, T34 and T54 confirm that customers, agents, administrators and public viewers each see exactly their permitted portion.

EQ2: real-time propagation without disclosure. Combining authenticated sockets, per-user and per-role rooms, and parcel rooms that require ownership or proof of the tracking code makes it possible to push changes immediately while confining each event to its legitimate audience. Agent positions are accepted only from the agent concerned and

Table 6.1: Assessment of objectives against evidence

ID	Assessment	Evidence and qualification	Open issues
O1	Achieved	Verification, lockout, rotation with reuse detection and revocation all verified by tests (T05–T22, T61–T62)	Password policy deviates from NIST (F12)
O2	Achieved	Role and ownership checks held in every attempted violation (Section 4.7)	Client guards are convenience only, as intended
O3	Largely achieved	Lifecycle, locking of final states and audit trail verified (T42–T56)	Forward skips allowed (F1)
O4	Achieved	Events reach authorised watchers only; public view is reduced (T34, T43, T49, T51)	Scalability not tested (F11)
O5	Partly achieved	QR identification at pickup and route maps work in the running system	No delivery-side confirmation; QR is not cryptographic proof; map service dependencies (F2, F3)
O6	Achieved within stated scope	Metrics, CSV/PDF export and user management verified (T24, T57–T62)	Analytics deliberately basic

forwarded only to watchers of that agent's active parcels. Tests T43, T49 and T51 confirm both delivery to authorised sockets and non-delivery to others.

EQ3: satisfaction of requirements. All 66 executed test cases passed. These cover every functional requirement that can be exercised through the API, together with the security, privacy and robustness requirements. The remaining requirements (QR scanning, status e-mail delivery, the bilingual interface and maps) were verified by inspection of the code and the running interface, not by automated assertions. Usability, scalability and behaviour in the field were not evaluated. The requirements are therefore satisfied for functional correctness and security on a single machine, with fourteen documented defects or findings, most of them minor.

6.5 Key Technical Contributions

1. A complete, documented courier management system for a single operator, covering registration through delivery with three role-specific interfaces, public tracking and a bilingual user interface.

2. A real-time design in which no event is broadcast globally and anonymous viewers must prove knowledge of a tracking code, so that live tracking does not leak data across customers.

3. A session design that applies refresh-token rotation with reuse detection, together with `tokenVersion` revocation, in a small web system, verified by adversarial tests.

4. A reproducible evaluation package: a black-box API and socket test harness that runs the unmodified backend against a disposable database, a screenshot capture script, and generated result tables. Future maintainers can rerun all of it.

6.6 Lessons Learned

- **Authorisation belongs on the server and next to the data.** The most serious early defects (role escalation at sign-up, IDOR on parcel reads, unauthenticated assignment and deletion) were fixed by moving checks into route guards *and* controllers. The tests show that both layers are needed.
- **Real-time features need the same care as REST.** Socket messages were initially trusted and broadcast globally. Treating them as untrusted input and scoping them to rooms was as important as securing the HTTP API.
- **Documentation drifts unless it is checked against code.** The audit for this report found several documents that described planned features as implemented. Writing claims only from code and executed tests produced a smaller but defensible account.
- **Evidence is cheap once automated.** A single harness that starts the real system against a disposable database gave repeatable evidence for dozens of requirements in under two minutes.

6.7 Recommendations for Future Work

Short term (corrective).

1. Resolve the defects in Table 4.5: remove the customer debug control, fix the currency symbol, consolidate the two public tracking pages, correct the QR page wording, guard `/parcel/:id/route`, and clear the lint errors.
2. Decide whether agents must pass through every lifecycle state and enforce the decision in `transitionError`.
3. Store geocoded coordinates on the parcel at booking and recompute routes only when endpoints change. Adopt a length-based password policy with breached-password screening.
4. Add the test harness to continuous integration and extend it with backend unit tests and frontend component tests.

Medium term (functional).

1. Delivery confirmation using a signed, single-use code (for example a one-time code sent to the recipient or a signed QR token) so that a delivery requires evidence from

the recipient.

2. Persistent location history and a shared Socket.IO adapter (for example Redis) to survive restarts and allow horizontal scaling.
3. Richer analytics: on-time rate, failure reasons, per-agent workload and time-series charts.
4. SMS notifications through a local gateway, and a progressive web app or native companion app for agents with background location.

Research directions. Once real operational data is available, the system could support the optimisation problems surveyed in the literature [2, 1], such as automatic agent assignment, route optimisation and data-driven delivery-time prediction. It could also support a controlled field study measuring whether live visibility reduces customer enquiries and failed deliveries.

6.8 Final Conclusion

SCPMS demonstrates that a small team can build, with widely used open-source web technologies, a courier management system in which responsibilities are explicit, every status change is recorded, live tracking is available to the right people and only to them, and the security mechanisms withstand the attacks most common in web applications. The evaluation supports these claims for functional correctness and security on a single machine, and it states plainly what remains unproven: usability, scalability and operational benefit in the field. The system is a sound foundation for those next steps, but it is not yet a production-ready, independently validated platform.

References

[1] M. Savelsbergh and T. Van Woensel, "50th anniversary invited article—City logistics: Challenges and opportunities," *Transportation Science*, vol. 50, no. 2, pp. 579–590, 2016.

[2] N. Boysen, S. Fedtke, and S. Schwerdfeger, "Last-mile delivery concepts: a survey from an operational research perspective," *OR Spectrum*, vol. 43, no. 1, pp. 1–58, 2021.

[3] L. Ranieri, S. Digiesi, B. Silvestri, and M. Roccotelli, "A review of last mile logistics innovations in an externalities cost reduction vision," *Sustainability*, vol. 10, no. 3, p. 782, 2018.

[4] J. Olsson, D. Hellström, and H. Pålsson, "Framework of last mile logistics research: A systematic review of the literature," *Sustainability*, vol. 11, no. 24, p. 7131, 2019.

[5] A. Musa, A. Gunasekaran, and Y. Yusuf, "Supply chain product visibility: Methods, systems and impacts," *Expert Systems with Applications*, vol. 41, no. 1, pp. 176–194, 2014.

[6] DHL, "Shipment tracking – unified API," DHL Developer Portal, https://developer.dhl.com/api-reference/shipment-tracking, accessed: Oct. 26, 2026.

[7] Pathao Ltd., "Pathao courier," Shopify App Store listing, https://apps.shopify.com/pathao-courier, accessed: Oct. 26, 2026.

[8] S. Lee, G. Tewolde, and J. Kwon, "Design and implementation of vehicle tracking system using GPS/GSM/GPRS technology and smartphone application," in *Proceedings of the 2014 IEEE World Forum on Internet of Things (WF-IoT)*, 2014, pp. 353–358.

[9] W. M. El-Medany, A. Al-Omary, R. Al-Hakim, and M. Nusaif, "Low cost real-time tracking system prototype using GM862 cellular quad band module," *Journal of Engineering, Design and Technology*, vol. 12, no. 3, pp. 389–407, 2014.

[10] W3C, "Geolocation API," W3C Recommendation, https://www.w3.org/TR/geolocation/, accessed: Oct. 19, 2026.

[11] I. Fette and A. Melnikov, "The WebSocket protocol," RFC 6455, Internet Engineering Task Force, 2011.

[12] M. Haklay and P. Weber, "OpenStreetMap: User-generated street maps," *IEEE Pervasive Computing*, vol. 7, no. 4, pp. 12–18, 2008.

[13] D. Luxen and C. Vetter, "Real-time routing with OpenStreetMap data," in *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, 2011, pp. 513–516.

[14] C. Larman and V. R. Basili, “Iterative and incremental development: A brief history,” *Computer*, vol. 36, no. 6, pp. 47–56, 2003.

[15] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.

[16] S. Wibbeling and F. Schneiders, “Research project ePOD@Home: Electronic proof of delivery at point of delivery,” in *Efficiency and Logistics*, ser. Lecture Notes in Logistics. Berlin, Heidelberg: Springer, 2013, pp. 99–108.

[17] A. Chaudhary, P. Chorge, S. Pathare, and S. Gaikwad, “Parcel Pulse: A web-based courier management system for enhanced operational efficiency and real-time tracking,” *International Journal of Scientific Research in Engineering and Management*, vol. 9, no. 11, 2025.

[18] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000.

[19] V. Pimentel and B. G. Nickerson, “Communicating and displaying real-time data with WebSocket,” *IEEE Internet Computing*, vol. 16, no. 4, pp. 45–53, 2012.

[20] S. Tilkov and S. Vinoski, “Node.js: Using JavaScript to build high-performance network programs,” *IEEE Internet Computing*, vol. 14, no. 6, pp. 80–83, 2010.

[21] Socket.IO, “Socket.IO documentation (v4): Rooms,” https://socket.io/docs/v4/rooms/, accessed: Oct. 19, 2026.

[22] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” *Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[23] OWASP Foundation, “OWASP top 10:2021,” https://owasp.org/Top10/, accessed: Oct. 19, 2026.

[24] A. Ron, A. Shulman-Peleg, and A. Puzanov, “Analysis and mitigation of NoSQL injections,” *IEEE Security & Privacy*, vol. 14, no. 2, pp. 30–39, 2016.

[25] M. B. Jones, J. Bradley, and N. Sakimura, “JSON web token (JWT),” RFC 7519, Internet Engineering Task Force, 2015.

[26] A. Barth, “HTTP state management mechanism,” RFC 6265, Internet Engineering Task Force, 2011.

[27] OWASP Foundation, “Cross-site request forgery prevention cheat sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html, accessed: Oct. 19, 2026.

[28] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, “Best current practice for OAuth 2.0 security,” RFC 9700 (BCP 240), Internet Engineering Task Force, 2025.

[29] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference*, Monterey, CA, USA, 1999, pp. 81–92.

[30] P. A. Grassi, J. L. Fenton, E. M. Newton, R. A. Perlner, A. R. Regenscheid, W. E. Burr, J. P. Richer, N. B. Lefkovitz, J. M. Danker, Y.-Y. Choong, K. K. Greene, and

M. F. Theofanos, “Digital identity guidelines: Authentication and lifecycle management,” National Institute of Standards and Technology, Tech. Rep. SP 800-63B, 2017.

[31] V. Agafonkin and Leaflet contributors, “Leaflet: An open-source JavaScript library for interactive maps,” https://leafletjs.com/, accessed: Oct. 19, 2026.

[32] OpenStreetMap Foundation, “Nominatim usage policy,” https://operations.osmfoundation.org/policies/nominatim/, accessed: Oct. 19, 2026.

[33] ISO/IEC, “ISO/IEC 18004:2015 information technology — automatic identification and data capture techniques — QR code bar code symbology specification,” International Organization for Standardization, 2015.

[34] ZXing contributors, “ZXing-js: Multi-format 1D/2D barcode image processing library,” https://github.com/zxing-js/library, accessed: Oct. 19, 2026.

[35] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.

[36] ISO/IEC, “ISO/IEC 25010:2011 systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models,” International Organization for Standardization, 2011.

[37] ISO/IEC/IEEE, “ISO/IEC/IEEE 29148:2018 systems and software engineering — life cycle processes — requirements engineering,” International Organization for Standardization, 2018.

[38] MongoDB, Inc., “MongoDB manual: TTL indexes,” https://www.mongodb.com/docs/manual/core/index-ttl/, accessed: Oct. 19, 2026.

[39] Helmet contributors, “Helmet: Help secure Express apps with HTTP headers,” https://helmetjs.github.io/, accessed: Oct. 19, 2026.

[40] OpenJS Foundation, “Express: Fast, unopinionated, minimalist web framework for Node.js,” https://expressjs.com/, accessed: Oct. 19, 2026.

[41] Mongoose, “Mongoose ODM documentation,” https://mongoosejs.com/docs/, accessed: Oct. 19, 2026.

[42] Meta Open Source, “React documentation,” https://react.dev/, accessed: Oct. 19, 2026.

[43] Vite, “Vite: Next generation frontend tooling,” https://vite.dev/, accessed: Oct. 19, 2026.

[44] Nodemailer, “Nodemailer documentation,” https://nodemailer.com/, accessed: Oct. 19, 2026.